

Optimizing LLM Inference: Fluid-Guided Online Scheduling with Memory Constraints

Ruicheng Ao*
Massachusetts Institute of Technology

Gan Luo†
Peking University

David Simchi-Levi*
Massachusetts Institute of Technology

Xinshang Wang‡
Alibaba Group

December 14, 2025

Abstract

Large Language Models (LLMs) is indispensable in today’s applications, but their inference procedure—generating responses by breaking text into smaller pieces and processing them using a memory-heavy element named Key-Value (KV) cache—requires a lot of computational resources, especially when memory is limited. This paper treats LLM inference optimization as a multi-stage online scheduling problem, where prompts arrive sequentially and the incremental expansion of the KV cache during inference renders conventional scheduling algorithms ineffective. To address this challenge, we develop a fluid dynamics approximation to establish a tractable benchmark, providing insights for devising effective scheduling algorithms. Building upon this foundation, we introduce the Waiting for Accumulated Inference Threshold (WAIT) algorithm as a warm-up. This method maintains multiple thresholds to determine the scheduling order of incoming prompts, optimizing resource utilization when output lengths are known at the time of arrival. In practical applications where output lengths are not known at the time of prompt arrival, we extend our method by introducing the Nested WAIT algorithm. This algorithm constructs a hierarchical framework comprising multiple segments, each defined by distinct thresholds, to effectively manage the random prompt arrivals with unknown output lengths. Theoretical analysis shows both algorithms have near-optimal performance compared with the fluid benchmark in the asymptotic regime, balancing throughput, latency, and Time to First Token (TTFT). Numerical experiments conducted with the Llama-7B model on an A100 GPU, utilizing both synthetic and real-world datasets, demonstrate that our approach achieves superior throughput and reduced latency compared to widely adopted baseline methods such as vLLM and Sarathi. This research bridges operations research and machine learning, presenting a theoretically grounded framework for the efficient deployment of large language models under memory constraints.

Keywords: Large Language Model, Key-value cache, Memory Constraint, Online scheduling

1 Introduction

Large Language Models (LLMs) are indispensable in natural language processing (NLP) (????????), supporting applications such as chatbots (????), search engines (??), and programming assistants (?). These models generate coherent text through *inference*—the process of producing responses to user queries—which imposes significant computational demands, particularly on memory and processing resources (?). Efficient inference scheduling is essential to optimize resource utilization, reduce operational costs, and minimize energy consumption (?). For instance, systems like ChatGPT incur daily inference costs exceeding \$700,000

*R. Ao and D. Simchi-Levi are with the Institute of Data, System and Society at Massachusetts Institute of Technology; emails: {aorc, dslevi}@mit.edu.

†G. Luo is with School of Mathematical Sciences, Peking University, Beijing, China; email: luogan@stu.pku.edu.cn.

‡X. Wang is with DAMO Academy, Alibaba US, Seattle, WA; email: xinshang.w@alibaba-inc.com.

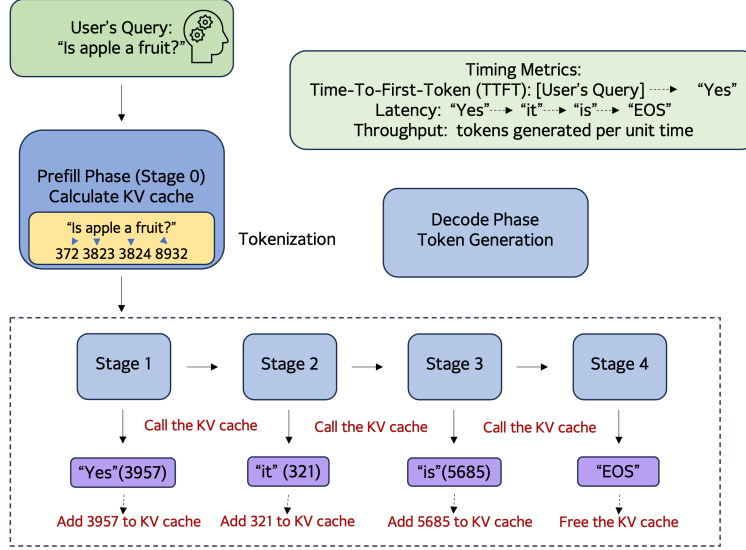


Figure 1: An example of LLM inference.

and contribute to substantial carbon emissions due to high electricity usage (??). Effective scheduling strategies can address these challenges by balancing system throughput and response latency (?).

Understanding the LLM inference process is essential for tackling its associated scheduling challenges. Figure ?? provides a step-by-step illustration of how a query is processed during inference. The process begins when a user submits a prompt—e.g., “Is apple a fruit?”—which is then passed through several computational stages. The inference pipeline consists of two main phases:

- **Prefill Phase (stage 0):** Upon receiving the prompt, the model first tokenizes the input into a sequence of discrete units (e.g., “Is”, “apple”, “a”, “fruit”, “?”). These tokens are then embedded and simultaneously processed in a single forward pass to compute the *Key-Value (KV) cache*, which stores intermediate representations (i.e., attention keys and values) for each token. These precomputed values are critical for enabling efficient reuse during subsequent decoding steps. This phase corresponds to Stage 0 in Figure ??, where all prompt tokens are embedded and their KV representations are added to the cache.
- **Decode Phase (stages 1 to l'):** After the prefill phase, the model enters the decode phase, where it generates the output one token at a time. At each stage, the model queries the existing KV cache to compute the next token, appends the new token to the output sequence, and updates the KV cache with its key-value pair. For instance, the model might first generate “Yes” (Stage 1), then “it” (Stage 2), followed by “is” (Stage 3), and finally the “End of Sequence” (EOS) token (Stage 4). This progression is depicted in the lower part of Figure ?. Each token generation step involves both reading from and writing to the KV cache, resulting in a memory footprint that grows linearly with the length of the generated sequence.

Figure ?? also highlights key performance metrics in LLM inference:

- **Time-To-First-Token (TTFT)** measures the latency from user input to the first generated token.
- **Latency** refers to the total time required to complete the generation of all output tokens.
- **Throughput** captures the average number of tokens generated per unit time.

This inference structure, particularly the growing memory requirements and sequential decoding pattern, introduces fundamental constraints on scheduling and batching. Efficient scheduling must account for both prompt heterogeneity and KV cache dynamics in order to optimize latency and resource utilization.

The KV cache is essential for efficiency, as it prevents the model from recalculating the attention history

for each new token. Without it, the computational cost would scale quadratically with the sequence length, making real-time inference infeasible. However, the expanding KV cache poses a significant scheduling challenge, as the memory footprint grows unpredictably during the decode phase. This variability, combined with stochastic prompt arrivals and differing response lengths, complicates resource allocation on hardware with finite capacity, such as GPUs. Exceeding memory limits can force the system to offload data to slower storage mediums, degrading performance and increasing latency (??).

The Eviction Challenge. A critical consequence of memory overflow is *prompt eviction*: when the total KV cache exceeds GPU memory capacity, the system must evict some in-progress prompts to make room for others. Evicted prompts lose their accumulated KV cache and must restart from the prefill phase, wasting all prior computation. This creates a vicious cycle: eviction frees memory temporarily, but restarted prompts consume resources again, potentially triggering further evictions. Importantly, *memory sufficiency alone does not prevent this failure mode*. Even when the total memory capacity C equals or exceeds the optimal steady-state memory M^* (as derived in our fluid analysis), naive scheduling policies like First-Come-First-Served (FCFS) can still trigger cascading evictions that reduce throughput by 12–25% (see Example ?? in Section ??). This observation motivates our central algorithmic insight: effective LLM inference scheduling requires not just sufficient capacity, but *eviction prevention through controlled admission*. Our threshold-based WAIT algorithms achieve this by temporarily holding prompts at each decoding stage boundary until memory conditions permit safe continuation.

Scheduling LLM inference tasks involves grouping prompts into *batches* processed concurrently on a GPU, combining prefill and decode phases to optimize resource use. This process is constrained by GPU memory limits, as the KV cache grows with each generated token, restricting the number of active prompts. Traditional scheduling methods, such as Shortest Job First (SJF), assume fixed job sizes and known processing times, making them not suitable for LLM inference where memory demands increase dynamically and output lengths are often unknown. For example, consider two prompts: one with a short input but a long, unpredictable output (e.g., “Summarize the history of artificial intelligence”), and another with a longer input but a short output (e.g., “Is 5 a prime number?”). In a traditional setting, SJF would prioritize the second prompt, expecting its shorter processing time to minimize average wait times. However, in LLM inference, the first prompt’s decode phase could generate hundreds of tokens, causing its KV cache to balloon over time and occupy significant memory, while the second prompt’s quick decode phase releases resources almost immediately after a longer prefill. Prioritizing the second prompt might reduce initial latency, but the first prompt’s prolonged memory usage could block other tasks, leading to GPU under-utilization and reduced throughput. Beyond SJF, other traditional methods like priority scheduling falter as they prioritize prompts without accounting for the KV cache’s unpredictable growth, potentially exhausting memory. While batching remains effective, it requires dynamic adaptation—e.g., adjusting batch sizes based on current memory usage—to accommodate the KV cache’s expansion, unlike static batching in traditional settings. These characteristics—stochastic prompt arrivals, multi-phase processing, and unpredictable resource needs—render classical operations research approaches inadequate, necessitating tailored scheduling strategies that account for dynamic memory growth and phase-specific demands.

Recent system-level optimizations for LLM inference (?????) focus on engineering solutions, such as batching and memory compression, but lack a rigorous mathematical foundation. In practice, output length predictions are imprecise or costly. For example, as shown in ?, the high-accuracy prediction is sensitive to the bucket size of output lengths for classification. When the output length is unknown, the performance of algorithms may degenerate significantly and performance guarantee obtained in full knowledge scenario no longer holds (see Proposition ?? in Section ?? as an example). To address this problem, we develop a fluid dynamics approximation to establish a tractable benchmark, providing insights for devising effective scheduling algorithms. Building upon this foundation, we introduce the Waiting for Accumulated Inference Threshold (WAIT) algorithm as a warm-up. This method maintains multiple thresholds to determine the scheduling order of incoming prompts, optimizing resource utilization when output lengths are known at the time of arrival. In practical applications where output lengths are not known at the time of prompt arrival, we extend our method by introducing the Nested WAIT algorithm. This algorithm constructs a hierarchical framework comprising multiple segments, each defined by distinct thresholds, to effectively manage the random prompt arrivals with unknown output lengths. It is able to “learn” its behavior as a prompt is

processed for more iterations. We show that both algorithms are asymptotically near-optimal, offering a theoretically grounded solution adaptable to practical settings.

1.1 Summary of Contributions

This work contributes to LLM inference scheduling through the following contributions:

- **Memory-Constrained Scheduling Model:** We develop a multi-stage online scheduling model that captures the dynamic growth of KV cache memory during LLM inference, where exceeding capacity triggers costly prompt evictions (Section ??). Using a fluid dynamics approximation from queueing theory, we characterize the optimal throughput and steady-state memory M^* under perfect scheduling (Section ??).
- **Eviction-Free Scheduling Algorithms:** We develop threshold-based algorithms that prevent memory overflow without sacrificing throughput. The WAIT algorithm (Section ??) demonstrates the core insight for known output lengths, while the *Nested WAIT* algorithm (Section ??) extends this to unknown output lengths through on-the-fly type classification. Both achieve asymptotic optimality via novel coupling arguments.
- **Experimental Validation:** We evaluate our algorithms on synthetic and real-world datasets, outperforming benchmarks like vLLM (?) and Sarathi (?), which are widely recognized high-performance inference engines, in average throughput using Llama2-7B on a single A100 GPU (Section ??).

These contributions bridge operations research and machine learning, providing the first theoretical framework for memory-constrained LLM inference scheduling that explicitly addresses the eviction problem.

1.2 Other Related Work

Online Scheduling Problem and Queueing System. Classical online scheduling problems focus on optimally assigning jobs that arrive sequentially, each with varying completion times. In settings with stochastic arrivals, foundational studies such as ????? have developed algorithms that learn arrival patterns or distributions to refine scheduling strategies over time. Beyond individual job processing, works like ???? have explored simultaneous batching and scheduling, grouping similar jobs to enhance efficiency. Within queueing theory, fluid models serve as first-order approximations of stochastic systems, enabling near-optimal control strategies ???? , while heavy-traffic analysis offers insights into long-term system behavior. However, these traditional approaches fall short in addressing the unique dynamics of LLM inference, where resource requirements are dynamic due to the growing KV cache and the inference process involves distinct prefill and decode phases, unlike the static, single-phase jobs or fixed number phases and resource occupancy in classical scheduling.

LLM Inference. Theoretical frameworks for analyzing Large Language Model (LLM) inference are scarce, despite their widespread use, as most prior work targets system-level optimizations rather than the unique scheduling demands of LLMs. Unlike traditional scheduling, LLM inference involves stochastic arrivals, dynamic memory constraints, and multi-phase processing, complicating analytical efforts. System-level works like ? (“Splitwise”) and ? (“DistServe”) split inference into prompt handling and pipeline execution, while scheduling methods such as ?, ?, and ? use batching to boost throughput—similar to our approach. Other optimizations, e.g., ? with multi-head latent attention and low-rank KV compression, or ? with Kendall’s Tau for prioritizing prompts by predicted output lengths, focus on engineering efficiency rather than giving theoretical insight. Moreover, when full knowledge of the arriving prompts is not available, it may lead to significant performance drops (see Proposition ?? in Section ??). Our work tries to tackle these challenges by developing a scheduling framework for the realistic case of unknown output lengths, using fluid dynamics approximations and coupling techniques from queueing theory to achieve asymptotic optimality, providing both practical relevance and a robust mathematical foundation. More recently, ? propose a stochastic processing model for LLM inference based on a linear batch processing time approach, similar to our own. They demonstrate that work-conserving scheduling algorithms—policies designed to keep resources active whenever tasks are available—achieve optimal throughput for both individual requests and multi-agent AI

workloads. Their framework extends to AI-agent scenarios such as parallel engines and fork-join models, highlighting its flexibility for complex LLM deployments. However, their analysis does not address the GPU memory usage of preempted or queued requests, particularly the memory needs of Key-Value caches for tasks awaiting processing. The study also does not explore theoretical aspects of latency metrics, such as time to first token (TTFT) or end-to-end request latency, which are important for real-time applications like interactive dialogue systems.

Concurrent Theoretical Work. A series of concurrent works by Zhou and collaborators (???) provide theoretical analyses of LLM inference scheduling with notable contributions. ? develop online scheduling algorithms for KV cache constraints with competitive ratio guarantees, while ? and ? extend the framework to variable-length requests and prediction uncertainty, respectively. A key modeling distinction is that these works adopt a *single-token-per-iteration* assumption (i.e., batch processing time $\tau = 1$ for one token generation), which abstracts away batching effects. In contrast, our model captures the fundamental property that iteration time scales with batch composition via Equation (??). This distinction has practical implications: in *prefill-decode disaggregated systems* (??)—where prefill and decode phases run on separate GPU clusters—the decode server processes only single-token generations, making their $\tau = 1$ assumption exact for the decode cluster. Our model accommodates both disaggregated and non-disaggregated deployments, with the linear iteration time model in Equation (??) being particularly accurate for decode-dominated batches. We believe these complementary perspectives enrich the theoretical understanding of LLM inference scheduling.

1.3 Notations

For integer $n \geq 1$, we denote $[n] = \{1, 2, \dots, n\}$ as the set of integers from 1 to n . For $x \in \mathbb{R}$, denote $\lceil x \rceil$ as the smallest integer not smaller than x and $\lfloor x \rfloor$ as the largest integer not greater than x . Denote $x_+ = \max\{x, 0\}$. For set S , denote $|S|$ as its cardinality. For two functions $f(T)$ and $g(T)$, we use $f(T) = O(g(T))$ if there exists constant $c_1 > 0$ such that $f(T) \leq c_1 g(T)$ as $T \rightarrow +\infty$ and $f(T) = \Omega(g(T))$ if there exists constant $c_2 > 0$ such that $f(T) \geq c_2 g(T)$ as $T \rightarrow +\infty$.

2 Model

This section formalizes the online scheduling problem for Large Language Model (LLM) inference under memory constraints. Unlike traditional scheduling, this task involves dynamically evolving resource demands due to the growing Key-Value (KV) cache and a two-phase (prefill and decode) processing structure. We first introduce the inference process, then describe prompt characteristics, batching rules, system constraints, and performance metrics.

2.1 Notation

We summarize the key notation used throughout this paper in Table ??.

2.2 Prompts and Inference Process

We analyze a system designed to process user queries, referred to as *prompts*, utilizing a single Graphics Processing Unit (GPU) for computation. We focus on maximizing the utilization of single GPU and our algorithms can be extended to multi-GPU settings such as those utilizing pipeline parallelism, where each prompts are processed on different GPU in different layers on LLM model. Additionally, when there is no communication cost, our algorithms can generalize naturally to this setting as well.

During the inference process, each prompt is divided into discrete units known as *tokens* during the prefill phase and generates output text through an inference process in the decode phase. A token represents a distinct text element, such as a word or subword, and its processing relies on a memory structure called the Key-Value (KV) cache. As noted in the introduction, the KV cache improves inference efficiency significantly by storing the computational history of prior tokens, thus avoiding redundant computations. The cache size increases as tokens are processed, requiring careful memory management to maintain system performance.

Table 1: Summary of Notation

Symbol	Description
m	Number of prompt types
j	Index of prompt type, $j \in \{1, \dots, m\}$
λ_j	Arrival rate of type- j prompts (Poisson process)
l_j	Input length (tokens) of type- j prompts after prefill
l'_j	Output length (tokens) of type- j prompts in decode phase
s	Stage index: $s = 0$ for prefill, $s \in \{1, \dots, l'_j\}$ for decode
n_{js}^t	Number of type- j prompts at stage s at time t
n_j	Threshold for type- j prompts in WAIT algorithm
C	GPU memory capacity
B^t	Batch of prompts processed at iteration t
τ	Iteration time to process a batch
d_0	Fixed overhead time per batch iteration
d_1	Time cost per unit of KV cache memory
M^*	Equilibrium memory usage in fluid model
n_j^*	Equilibrium number of active type- j prompts
Throughput*	Equilibrium throughput (fluid benchmark)
Π	Class of admissible scheduling policies

Prompt Characteristics: Prompts arrive stochastically and are classified into m distinct types according to their input / output lengths, indexed by $j \in \{1, \dots, m\}$. We assume that prompts of type j arrive according to a Poisson process with rate λ_j . A prompt of type j has the following properties:

$$\begin{cases} \text{a length of } l_j \text{ tokens after prefill;} \\ \text{a length of } l_j + l'_j \text{ tokens after decode.} \end{cases}$$

Therefore, a type- j prompt must undergo one prefill iteration followed by l'_j decode iterations to complete its inference processing. During the prefill phase, the prompt’s input context and question are embedded as l_j tokens in a single iteration, consuming a KV cache of size l_j units. In the subsequent decode phase, one output token is generated per iteration, each incurring an additional unit of KV cache memory. For clarity of exposition, we define a prompt to be in **stage** $s = 0$ if it is about to undergo the prefill phase, and in **stage** s (where $s \in \{1, \dots, l'_j\}$) if it is scheduled to receive its s -th output token during the decode phase. Thus, a prompt at stage s occupies memory of size $l_j + s$ units. Though the total number of different types can be large (for example, from 1 to 10000), as we will demonstrate later in this paper, our algorithms and their theoretical guarantees exhibit only a weak (logarithmic) dependence on m , relying primarily on the total arrival rate across all types. This dependence on the total arrival rates ensures that our approach scales efficiently across various operational scenarios, regardless of the number of types. Moreover, while our analysis focuses on fixed arrival rates λ_j , our threshold-based algorithms can be extended to settings with time-varying arrivals by dynamically adjusting the thresholds based on the arrival rates.

2.3 Batching and GPU Processing Dynamics

To optimize resource utilization, the GPU processes tasks in *batches*, combining prompts for prefilling and prefilled prompts for decoding. Figure ?? illustrates the temporal dynamics of batching and scheduling in GPU-based inference. Two prompts, $P1$ and $P2$, arrive sequentially and are processed according to their current stage—prefill or decode—at each time step. Initially, $P1$ arrives and enters the prefill phase, embedding its input sequence “How are you?” in a single iteration. Shortly after, $P2$ arrives with the query “Is 5 a prime?”, which is also scheduled for prefill. The GPU batches prompts in the same phase together; thus, $P1$ is processed for prefill alone, followed by $P2$. At the next step, $P1$ enters the decode phase, generating its first output token “I’m,” while $P2$ undergoes prefill in parallel. Subsequently, both prompts advance in the decode phase, each generating one token per iteration: “Yes” for $P2$ and “I’m fine” for $P1$. This example highlights how the scheduler interleaves prompts in different stages—prefill and decode—into

shared GPU batches, thereby improving throughput. Importantly, prefill stages involve initializing KV caches based on input tokens, while decode stages extend outputs one token at a time, with incremental KV cache updates.

In general, consider a batch composed of:

- n_1 prompts in the prefill phase (stage $s = 0$), where prompt i has input length $l_{j(i)}$ tokens corresponding to its type $j(i)$. These prompts are processed to initialize their KV caches.
- n_2 prompts in the decode phase, where prompt i is at stage $s_i \in \{1, \dots, l'_{j(i)}\}$, having generated s_i output tokens. The total KV cache size for prompt i is $l_{j(i)} + s_i$ tokens.

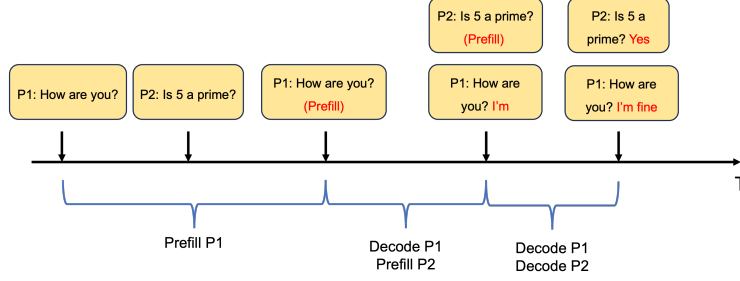


Figure 2: Example of batching and scheduling.

Based on experimental results from ???, we model the *iteration time* to process a batch as:

$$\tau = d_0 + d_1 \cdot \underbrace{\left(\sum_{i=1}^{n_1} l_{j(i)} + \sum_{i'=1}^{n_2} (l_{j(i')} + s_{i'}) \right)}_{\text{Total KV cache size in batch}}, \quad (1)$$

where d_0 denotes the fixed overhead time associated with processing a batch, and d_1 represents the time cost per unit of KV cache memory. This formulation indicates that the iteration time for each batch scales linearly with the total memory utilization, reflecting the direct dependence of computational latency on memory consumption involving reading existing KV cache and computing the new one. Figure ?? provides a numerical validation of this modeling. We deploy Llama-7B and Llama-13B on a single L20 GPU to test time cost per-iteration, using batches consisting of different number of tokens with 20 repeats. The result is highly in accordance with our model.

[Applicability of the Linear Model] The linear iteration time model (??) is particularly accurate in settings where: (i) batches are decode-dominated, as each decode iteration processes one token per prompt and the computational cost is dominated by KV cache memory access; (ii) the system employs prefill-decode disaggregation (??), where the decode server handles only decode operations, making Eq. (??) exact. In mixed prefill-decode batches, the linear model provides a good approximation when prefill lengths are similar across prompts.

2.4 Constraints and Performance Metrics

Memory Constraint: At every batch iteration t , the total memory usage must not exceed GPU capacity C :

$$\sum_{i \in B^t} (l_{j^t(i)} + s_i^t) \leq C, \quad (2)$$

where B^t is the batch set at iteration t , $j^t(i)$ is the type of prompt i , and $s_i^t \in \{0, 1, \dots, l'_{j^t(i)}\}$ is its current stage. Crucially, unlike traditional scheduling where job sizes are fixed, the memory footprint $l_{j^t(i)} + s_i^t$ grows *dynamically* as decode progresses (s_i^t increases each iteration). This dynamic growth creates the risk of *eviction*: if memory becomes insufficient, ongoing prompts may need to be preempted and their KV caches discarded, leading to wasted computation and potential cascading failures.

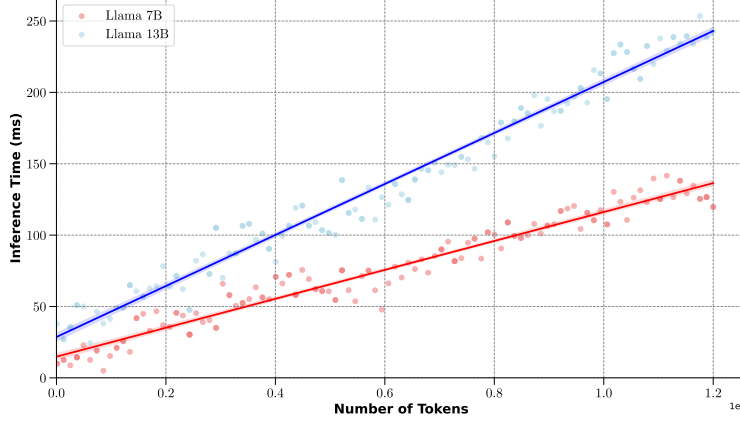


Figure 3: Inference time of Llama-7B and Llama-13B with different number of tokens in the batch on a single L20 GPU

Performance Metrics: We define clearly the metrics guiding scheduling optimization:

- **Throughput:** Average number of tokens generated in decode phase per unit time, measuring system efficiency. The throughput for a prompt is counted only after it is completed to ensure validity of the output.
- **Latency:** Average elapsed time from prompt arrival to completion, capturing responsiveness.
- **Time to First Token (TTFT):** Average delay from prompt arrival to the generation of the first token, reflecting initial user-perceived delay.

2.5 Optimization Problem

We formalize the problem as optimizing throughput subject to latency and TTFT constraints over continuous time horizon $[0, T]$:

$$\begin{aligned}
& \max_{\pi \in \Pi} \quad \mathbb{E} \left[\text{Throughput}^{(T, \pi)} \right], \\
& \text{s.t.} \quad \mathbb{E} \left[\text{Latency}^{(T, \pi)} \right] \leq L^T, \\
& \quad \mathbb{E} \left[\text{TTFT}^{(T, \pi)} \right] \leq F^T, \\
& \quad \sum_{i \in B^t} (l_{j^t(i)} + s_i^t) \leq C, \quad \forall t,
\end{aligned} \tag{3}$$

where Π denotes the class of admissible scheduling policies that are *non-anticipating* (decisions depend only on past and current information) and *work-conserving* (the GPU is not idle when prompts are waiting). Preemption—pausing and later resuming ongoing prompts—is allowed. However, due to substantial overhead costs incurred from KV cache recomputation, we design policies that hold computed KV caches on GPU while temporarily pausing certain prompts, thereby avoiding costly evictions.

3 Fluid Dynamics and Equilibrium

To design an effective scheduling policy for LLM inference, we first establish a theoretical benchmark representing the system’s optimal performance under idealized, steady-state conditions. This section develops a *fluid model* that approximates the stochastic, discrete prompt arrivals and processing with continuous, deterministic flows, where the arrivals and outputs are in equilibrium.

Fluid models are widely used in operations research to study online scheduling, queueing systems, resource allocation by transforming random, discrete events into continuous flows. In LLM inference, prompts arrive

stochastically (following a Poisson process with rate λ_j for type j), and processing consumes significant GPU memory due to KV caches. A fluid model provides a tractable framework to analyze the system's equilibrium state, where prompt arrival rates balance completion rates and memory usage stabilizes. This benchmark quantifies the ideal throughput achievable under memory constraints.

Consider a system processing prompts of types $j \in \{1, \dots, m\}$, each arriving at rate λ_j , with input length l_j (tokens processed in the prefill phase) and output length l'_j (tokens generated in the decode phase). We build this model by starting with a single type and extending to more general scenarios with multiple types of prompts.

3.1 Single-Type Fluid Model

We begin with a system handling prompts of a single type j . Each prompt occupies memory based on its stage: l_j tokens in the prefill stage ($s = 0$) and $l_j + s$ tokens in the s -th decode stage ($s = 1, \dots, l'_j$). Note that when the prompt completes stage $s = l'_j$, it finishes decoding and its KV cache is cleared from GPU memory. In equilibrium, the steady-state number of active prompts is n_j^* (i.e., prompts being processed in each batch). By allowing the prompt numbers to be continuous, we can, without loss of generality, assume that for each stage $s \in \{0, 1, \dots, l'_j\}$, there are exactly $n_j^*/(l'_j + 1)$ prompts at stage s in the batch, which not only simplifies the dynamics but provides the most balanced distribution across stages in equilibrium. As a result, for each stage s , there are $n_j^*/(l'_j + 1)$ prompts of type j requiring memory of size $l_j + s$, while the KV cache of prompts completing stage l'_j is cleared after the iteration. Additionally, we assume that $\lambda_j d_0(l'_j + 1) \left(l_j + \frac{l'_j}{2}\right) < 1$, otherwise, the latency will grow to infinity.

Proposition 1 *When the condition $\lambda_j d_0(l'_j + 1) \left(l_j + \frac{l'_j}{2}\right) \geq 1$ is satisfied, the system becomes unstable in the sense that, the expected average latency under any scheduling policy with deterministic arrival rate λ_j grows linearly with time. Formally,*

$$\mathbb{E}[\text{Latency}^{(T, \pi)}] = \Omega(T) \quad \text{as } T \rightarrow \infty.$$

To see intuition for this result, observe that completing a single prompt requires a total processing time of $d_1 \sum_{k=0}^{l'_j} (l_j + k) = d_1(l'_j + 1) \left(l_j + \frac{l'_j}{2}\right)$ for its key-value (KV) cache across all processing stages. Each iteration also incurs an additional fixed overhead time d_0 . The condition $\lambda_j d_0(l'_j + 1) \left(l_j + \frac{l'_j}{2}\right) \geq 1$ implies that the product of the prompt arrival rate λ_j and the processing time per prompt is at least 1, indicating that the system's processing capacity is insufficient to keep pace with incoming prompts. As a result, the queue of pending prompts grows over time, causing the expected average latency to increase linearly with T .

By direct calculation, the total memory usage is given by:

$$M_j^* = \frac{n_j^*}{l'_j + 1} \sum_{k=0}^{l'_j} (l_j + k) = n_j^* \left(l_j + \frac{l'_j}{2}\right). \quad (4)$$

While in equilibrium, the arrivals during each iteration should equal the number of completions, i.e. the number of prompts in stage l'_j . By our model assumption (??), we have

$$\underbrace{(d_0 + d_1 M_j^*)}_{\text{Time per iteration}} \lambda_j = \left(d_0 + d_1 n_j^* \left(l_j + \frac{l'_j}{2}\right)\right) \lambda_j = \frac{n_j^*}{l'_j + 1}$$

Thus

$$n_j^* = \frac{d_0 \lambda_j (l'_j + 1)}{1 - d_1 \lambda_j (l'_j + 1) \left(l_j + \frac{l'_j}{2}\right)}.$$

It follows that the equilibrium memory occupancy M_j^* is given by

$$M_j^* = n_j^* \left(l_j + \frac{l'_j}{2} \right) = \frac{d_0 \lambda_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right)}{1 - d_1 \lambda_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right)} \quad (5)$$

The average throughput at equilibrium is thereby the number of prompts in each batch divided by the time consumption for one iteration:

$$\mathbf{Throughput}^*_j = \frac{n_j^*}{d_0 + d_1 n_j^* \left(l_j + \frac{l'_j}{2} \right)}, \quad (6)$$

as each completed prompt generates l'_j tokens. This establishes the relationship between arrival rate, memory, and throughput for a single prompt type under equilibrium, where the arrivals equal the completions.

3.2 Multiple-Type Fluid Model

We now extend our fluid dynamics in equilibrium to multiple types $j \in \{1, \dots, m\}$, each with arrival rate λ_j , output length of l_j after prefill phase, and additionally l'_j output length after decode phase. We assume that the system will maintain n_j^* active prompts per type in equilibrium. We can calculate the total memory utilization in equilibrium following the derivation of (??):

$$M^* = \sum_{j=1}^m n_j^* \left(l_j + \frac{l'_j}{2} \right) \quad (7)$$

with:

$$\underbrace{(d_0 + d_1 M^*)}_{\text{Time per iteration}} \lambda_j = n_j^*, \quad \forall j \in \{1, 2, \dots, m\}$$

since the arrivals during each iteration should equal the number of completions in equilibrium. Substituting the above equations into (??) yields

$$M^* = \frac{d_0 \sum_{j=1}^m \lambda_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right)}{1 - d_1 \sum_{j=1}^m \lambda_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right)} \quad (8)$$

Aggregating the results above, we get the processing time per iteration in equilibrium:

$$\begin{aligned} & \text{Processing Time}^* \\ &= d_0 + d_1 M^* \\ &= \frac{d_0}{1 - d_1 \sum_{j=1}^m \lambda_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right)}, \end{aligned} \quad (9)$$

as well as the average throughput:

$$\begin{aligned} & \mathbf{Throughput}^* \\ &= \frac{1}{d_0} \left(\sum_{j=1}^m n_j^* \right) \cdot \left(1 - \sum_{j=1}^m \lambda_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right) \right) \\ &= \sum_{j=1}^m \lambda_j (l'_j + 1) \end{aligned} \quad (10)$$

The equilibrium throughput serves as a benchmark for evaluating the theoretical performance of any non-predictive policy within the policy class Π . The following proposition demonstrates that the expected throughput of any policy in Π is bounded above by the equilibrium throughput $\mathbf{Throughput}^*$, as specified in Equation (??).

Proposition 2 Suppose that the memory capacity satisfies $C \geq M^*$, where M^* denotes the equilibrium memory given in (??). Then, for any online policy $\pi \in \Pi$, the expected throughput satisfies

$$\mathbb{E}[\text{Throughput}^{(T,\pi)}] \leq \text{Throughput}^*,$$

where Throughput^* is given in (??).

Thus, for any non-predictive policy without preemption, the expected average throughput cannot exceed Throughput^* . Additionally, the equilibrium throughput, expressed as $\sum_{j=1}^m \lambda_j(l'_j + 1)$, characterizes the system's optimal steady-state performance under the memory constraint C . This benchmark not only guides the design of scheduling policies in subsequent sections but also facilitates the evaluation of how closely practical strategies can approximate this optimal performance under stochastic conditions.

4 Scheduling with Known Arrival Types: The WAIT Algorithm

In this section, as a warm-up, we introduce the *Waiting for Accumulated Inference Threshold* (WAIT) algorithm, a scheduling policy designed to maximize throughput in LLM inference systems under known prompt arrival types, while adhering to memory constraints and performance metrics including latency and time-to-first-token (TTFT). Leveraging the fluid dynamics established in Section ??, our WAIT tries optimizes batch formation and scheduling by accumulating prompts until specific thresholds are met, ensuring efficient resource utilization in stochastic environments. We assume that the memory capacity $C \geq M^*$, the equilibrium memory capacity given in (??) (otherwise the throughput can never approach the optimal fluid throughput (??) by Proposition ??).

4.1 Algorithm Overview

The WAIT algorithm operates by maintaining m inventories of prompts and tracking their current inventory across all types and stages until the system reaches a state in accordance with the fluid dynamics. For each prompt type $j \in [m]$ and stage $s \in \{0, 1, \dots, l'_j\}$, the algorithm sets a threshold n_j for type j prompts, where n_j is derived from the fluid dynamics presented in Section ?? and will be detailed subsequently.

At each continuous time t , the WAIT algorithm monitors the current inventory n_{js}^t , defined as the number of waiting prompts of type j at stage s , for $s = 0, 1, \dots, l'_j$, alongside stochastic arrivals following Poisson distribution $\text{Poisson}(\lambda_j)$ in each unit time interval. At time t , the algorithm incorporates type j prompts into the batch for a new iteration whenever the inventory level satisfies:

$$n_{j0}^t \geq n_j.$$

This condition implies that type j prompts are processed at time t only if the number of type j prompts awaiting prefill is at least n_j . When this criterion is met, WAIT constructs a batch B^t by selecting $\min\{n_j, n_{js}^t\}$ prompts of type j at each stage $s = 0, 1, \dots, l'_j$ for all j that satisfy the threshold condition. This batch is then processed in the current iteration, utilizing GPU memory proportional to the total number of tokens processed. Following processing, the inventory is updated, and prompts progress to their subsequent stages as appropriate. If the condition is not satisfied for any j , the algorithm pauses, permitting additional prompts to accumulate in the next time step. Moreover, the algorithm preserves the KV caches computed for waiting prompts throughout this process, thereby avoiding redundant computations. This threshold-based strategy ensures that the batch composition aligns with the fluid dynamics, striving to minimize memory waste and enhance throughput in the asymptotic regime, which will be elaborated later. The complete specification of the WAIT algorithm is presented in Algorithm ?. Figure ?? shows how WAIT works at batching.

4.2 Asymptotic Analysis

We evaluate the performance of WAIT in an asymptotic regime that captures long-run system behavior. Specifically, we scale the arrival rates to $\lambda_j^{(\zeta)} = \zeta \lambda_j$ and processing speeds accordingly $((d_0^{(\zeta)}, d_1^{(\zeta)}) = (\zeta^{-1}d_0, \zeta^{-1}d_1))$ while keeping memory capacity C fixed. This can equivalently be viewed as scaling the time

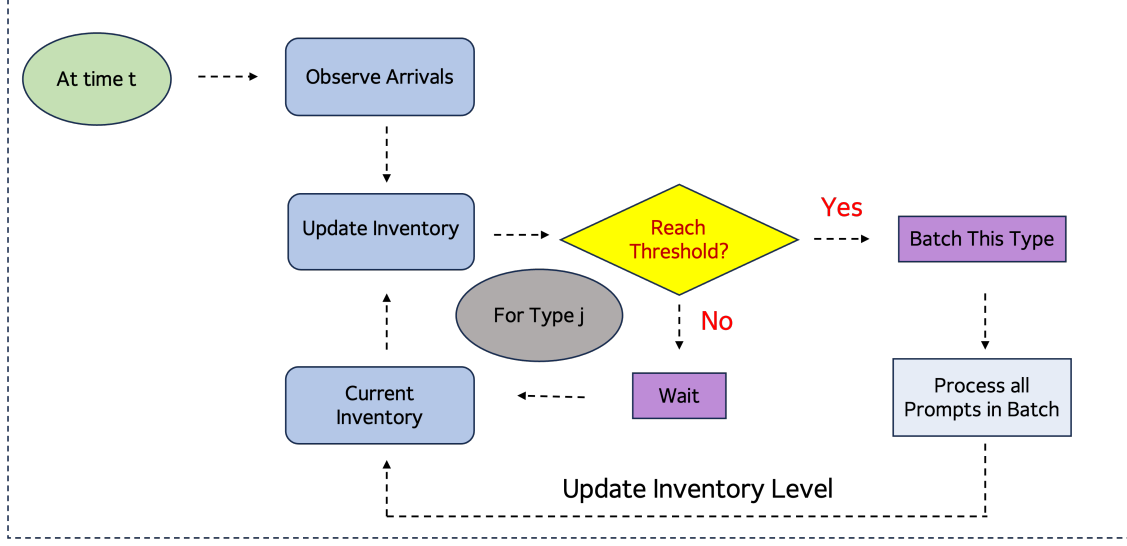


Figure 4: Pipeline of Algorithm ??

horizon from T to ζT while keeping arrival rates and processing speeds unchanged. As $\zeta \rightarrow \infty$, we obtain infinite-horizon limits that characterize the system’s long-run average performance, allowing us to study throughput stability under sustained load.

[Clarification on Terminology] Our asymptotic regime differs from traditional heavy-traffic limits in queueing theory, which typically involve load approaching the boundary of the stability region. Here, the load is fixed and we study infinite-horizon limits. We adopt the term “asymptotic regime” to avoid confusion.

Performance Metric: We focus on the average throughput, denoted $\mathbf{Throughput}^{(\zeta, \pi)}$, where π represents the WAIT policy, defined as:

$$\mathbf{Throughput}^{(\zeta, \pi)} = \frac{1}{\zeta T} \sum_{j=1}^m N_j^{(\zeta, \pi)},$$

with $N_j^{(\zeta, \pi)}$ being the total number of type- j tokens processed under scaling ζ . The average latency $\mathbf{Latency}^{(\zeta, \pi)}$, $\mathbf{TTFT}^{(\zeta, \pi)}$ and TTFT are defined similarly by dividing the quantity by ζ . Our objective is to demonstrate that $\mathbf{Throughput}^{(\zeta, \pi)}$ approaches the fluid equilibrium benchmark $\mathbf{Throughput}^* = \sum_{j=1}^m (1 + l'_j) \lambda_j$ (??) as $\zeta \rightarrow \infty$ while giving bound to the latency and TTFT.

4.3 Theoretical Analysis

When the memory constraint satisfies $C \geq M^*$, consider the WAIT policy π , parameterized by $[n_1, \dots, n_m]$, where n_j represents the threshold for the number of type j prompts at each stage. We will demonstrate that the WAIT policy is asymptotically optimal when the thresholds meet the following inequalities:

$$\begin{aligned} \Delta T(n_{1:m}) &:= d_0 + d_1 M^\pi \leq \frac{n_j}{\lambda_j}, \forall j \in [m], \\ M^\pi &= \sum_{j=1}^m n_j (l'_j + 1) (l_j + \frac{l'_j}{2}) \end{aligned} \tag{11}$$

where M^* is defined in Equation (??). To understand the intuition, one can refer to a derivation similar to that in Equation (??) and note that, if type j is included in the current iteration, the expected number of arrivals of type j during this iteration will not exceed n_j . This indicates that the system’s behavior closely approaches the fluid equilibrium. Formally, our main theorem is presented below.

Algorithm 1 WAIT: Waiting for Accumulated Inference Threshold

Input: Memory C , arrival rates λ_j , thresholds n_j satisfying (??), $\forall j \in [m]$

```

1: Initialize prompt inventory  $n_{js} \leftarrow 0$  for all  $j \in [m], s \in \{0, 1, \dots, l'_j\}$ 
2: Initialize event queue with arrival events for each type  $j$ 
3: Set current time  $t \leftarrow 0$ 
4: while True do
5:   Wait for the next event (arrival or batch completion)
6:   if event is an arrival of type  $j$  then
7:     Update inventory:  $n_{j0} \leftarrow n_{j0} + 1$  ▷ Add new prompt to waiting queue of prefill phase
8:   else if event is a batch completion then
9:     Update inventory: For each  $j$  in batch  $B$ ,  $n_{js} \leftarrow n_{js} - \min\{n_j, n_{js}\}$  for  $s = 0, \dots, l'_j$ 
10:    Advance prompts: For each  $j$ , move  $\min\{n_j, n_{js}\}$  prompts from stage  $s$  to  $s + 1$ 
11:    Clear KV caches for prompts reaching stage  $l'_j + 1$  ▷ Free memory for completed prompts
12:  end if
13:  Check if  $n_{j0} \geq n_j$  for any  $j \in [m]$  ▷ Check threshold for batching
14:  if condition  $n_{j0} \geq n_j$  is met for some  $j$  then
15:    Form batch  $B$  by selecting  $\min\{n_j, n_{js}\}$  prompts of type  $j$  at each stage  $s$  for all  $j$  meeting the condition
    ▷ Create batch
16:    Process batch  $B$ , keep prompts outside the batch waiting and donot delete their KV caches
17:  end if
18: end while

```

Theorem 1 Assume the thresholds $\pi = [n_1, \dots, n_m]$ satisfy (??), then we have

$$\begin{aligned} \text{Throughput}^* - \mathbb{E} \left[\text{Throughput}^{(\zeta, \pi)} \right] &= O((\zeta T)^{-\frac{1}{2}}), \\ \mathbb{E} \left[\text{Latency}^{(\zeta, \pi)} \right], \mathbb{E} \left[\text{TFTT}^{(\zeta, \pi)} \right] &= O\left((\zeta T)^{\frac{1}{2}}\right). \end{aligned}$$

Moreover, when we have $\Delta T_{[1, \dots, m]} < n_j / \lambda_j, \forall j \in [m]$ where $\Delta T_{[1, \dots, m]}$ is defined in (??), it holds that

$$\begin{aligned} \text{Throughput}^* - \mathbb{E} \left[\text{Throughput}^{(\zeta, \pi)} \right] &= O(1/(\zeta T)), \\ \mathbb{E} \left[\text{Latency}^{(\zeta, \pi)} \right], \mathbb{E} \left[\text{TFTT}^{(\zeta, \pi)} \right] &= O(1). \end{aligned}$$

Proof Sketch. The proof proceeds in three main steps:

Step 1: Dimensionality Reduction via Thresholds. The key insight is that WAIT's threshold mechanism decouples the multi-stage, multi-type dynamics. For each type j , the threshold condition $n_{j0}^t \geq n_j$ ensures that whenever type j enters a batch, the expected arrivals during that iteration do not exceed completions. This transforms the complex joint dynamics into m independent queue analyses.

Step 2: Coupling Construction. For each type j , we construct a dominating process $\{\tilde{W}_j^s\}$ indexed by batch number s :

$$\tilde{W}_j^0 = 2n_j, \quad \tilde{W}_j^{s+1} = \max\{2n_j, \tilde{W}_j^s + X_j^s - n_j\},$$

where $X_j^s \sim \text{Poisson}(\lambda_j \cdot \Delta T_{[1, \dots, m]})$ represents arrivals during iteration s . This Lindley-type recursion upper bounds the actual queue length W_j^s at each iteration. The threshold condition (??) ensures $\mathbb{E}[X_j^s] \leq n_j$, giving zero or negative drift.

Step 3: Performance Bounds. When $\Delta T_{[1, \dots, m]} = n_j / \lambda_j$ (zero drift), standard random walk analysis yields $\mathbb{E}[\tilde{W}_j^S] = O(\sqrt{S})$, translating to throughput gap $O((\zeta T)^{-1/2})$. When $\Delta T_{[1, \dots, m]} < n_j / \lambda_j$ (negative drift), Kingman's bound gives $\mathbb{E}[\tilde{W}_j^S] = O(1)$, yielding throughput gap $O((\zeta T)^{-1})$ and $O(1)$ latency. The detailed proof is in Appendix.

It is noteworthy that the results concerning different performance metrics in Theorem ?? match the worst-case lower bounds for latency and Time to First Token (TTFT). See the following proposition.

Proposition 3 *There exists an instance where $C = M^*$, and for any non-predictive online policy π , the expected average latency and TTFT satisfy*

$$\mathbb{E}[\text{Latency}^{(\zeta, \pi)}], \quad \mathbb{E}[\text{TTFT}^{(\zeta, \pi)}] = \Omega((\zeta T)^{1/2}).$$

Moreover, we demonstrate that the latency for type j prompts depends exclusively on the relationship between $\Delta T_{[1, \dots, m]}$ and n_j/λ_j . Specifically, when $\Delta T_{[1, \dots, m]} < n_j/\lambda_j$, the average latency and TTFT for type j prompts are $O(1)$, offering a more precise performance estimate than that provided in Theorem ??.

Another interesting result is that, the First-Come-First-Serve (FCFS) policy, which processes arriving prompts in the order of their arrival as long as memory permits, will have poor theoretic performance. See the proposition below.

Proposition 4 *When the memory capacity C equals the equilibrium memory M_j^* , there exists an instance where the First-Come-First-Serve (FCFS) policy described above incurs a throughput gap of $\text{Throughput}^* - \mathbb{E}[\text{Throughput}^T] = \Omega(1)$ as $T \rightarrow \infty$.*

By Proposition ??, when the memory capacity equals the equilibrium memory M_j^* , we can see that the FCFS can never reach asymptotically optimal performance. Indeed, this myopic policy may have to frequently remove ongoing prompts to accommodate new arrivals and require those preempted prompts to restart subsequently, which yields its sub-optimal performance in throughput.

To illustrate this phenomenon concretely, consider the following example of **FCFS cascading failure**:

[Eviction Cascade under FCFS] Consider a single prompt type with prefill length $l = 1$ and decode length $l' = 1$. Each prompt occupies 1 unit of memory after prefill and 2 units after decode. Let the memory capacity be $C = 12$ and arrival rate $\lambda = 4$. The fluid equilibrium has 4 prompts at each stage (prefill and decode), using exactly $4 \times 1 + 4 \times 2 = 12$ units of memory with 4 completions per iteration.

Now suppose the system starts slightly imbalanced with 6 prompts at stage 0 (awaiting prefill) and 3 prompts at stage 1 (in decode). Under FCFS:

1. **Iteration 1:** Process all 9 prompts. The 3 decode prompts complete, but the 6 prefill prompts advance to decode, requiring $6 \times 2 = 12$ units. Memory is exactly full with only 3 completions.
2. **Iteration 2:** During iteration 1, approximately 4 new prompts arrive. To admit them for prefill ($4 \times 1 = 4$ units needed), we must evict 2 decode prompts (freeing $2 \times 2 = 4$ units). Now we have 4 prefill + 4 decode = $4 + 8 = 12$ units.
3. **Iteration 3:** The 4 decode prompts complete (4 completions), but we also re-admit the 2 evicted prompts which must restart from prefill. This creates imbalance again.

This eviction-restart cycle persists, yielding approximately 3-3.5 completions per iteration instead of the optimal 4. **Even though $C = M^* = 12$ (theoretically sufficient capacity), FCFS causes cascading failures that reduce throughput by 12-25%.**

This example demonstrates a key insight: *memory sufficiency alone does not guarantee stability*. The WAIT algorithm avoids such cascading failures by using thresholds to maintain balanced inventory across stages, ensuring the system stays close to the fluid equilibrium.

A limitation of WAIT is its reliance on known arrival types and rates, which may not hold in realistic environments (though we can approximate it by using predictors, see e.g. (?)). In the next section, we address this issue by extending WAIT to handle unknown arrival types, introducing a nested variant that adapts to stochastic settings with unknown prompt types upon arrival.

5 Scheduling with Unknown Arrival Types: The Nested WAIT Algorithm

In the previous section, we introduced the WAIT algorithm for scheduling inference tasks with known prompt types, achieving near-optimal throughput. However, real-world LLM inference systems often encounter

prompts whose types—and consequently their output lengths—are unknown upon arrival. To address this challenge, we propose the *Nested Waiting for Accumulated Inference Threshold* (Nested WAIT) algorithm, which extends the WAIT framework to manage stochastic arrivals with unknown types.

5.1 Algorithm Overview

To begin with, note that adapting algorithms for known lengths by assuming worst-case lengths is inefficient. For example, with two prompt types (input length 1, arrival rates equal), one with output length 1 and another 10,000, assuming 10,000 for all yields memory utilization as low as $1/5000$. Therefore, the Nested WAIT algorithm extends the WAIT policy by introducing a nested structure to manage prompts with unknown output lengths. The mechanism tries to distinguish different types of arrivals by iterating step by step, rather than assuming or guessing this information at the beginning.

We assume that while the specific type of each arriving prompt is unknown, the possible output lengths follow a known distribution, inferred from the inference service. Formally, let the output length of type j be l_j after the prefill phase and l'_j after the decode phase, with $l'_1 < l'_2 < \dots < l'_m$. The algorithm organizes the inference process into m nested segments, each corresponding to cumulative processing stages up to these potential output lengths. It is parameterized by m thresholds $\pi = [n_1, \dots, n_m]$, where n_i corresponds to the i -th segment. The core logic determines prompt types upon completing each segment.

The process divides into m segments. In the first segment, the total arrival rate is $\lambda_1 + \dots + \lambda_m$, covering prompts at stages $0, 1, \dots, l'_1$. After $l'_1 + 1$ iterations, all prompts in segment 1, except those of type 1, advance to the second segment, spanning stages $l'_1 + 1, \dots, l'_2$. After $l'_2 - l'_1$ iterations, all prompts in segment 2, except those of type 2, proceed to segment 3. This continues for subsequent segments. If the number of prompts for the i -th segment is insufficient, processing pauses for that segment and all subsequent ones, retaining their KV caches on the GPU for future use. In that case, we will batch exact n_j prompts at each stage in segment j , $\forall 0 \leq j \leq i - 1$ and perform one iteration. Since there are new arrivals with output length not smaller than l_i in future, we expect segment i to resume joining in the batch very soon. Algorithm ?? formalizes this procedure, and Figure ?? illustrates the pipeline. For simplicity, we assume $l_1 = l_2 = \dots = l_m$ (identical prefill output lengths), though the general case extends similarly to Algorithm ??: segment groups are formed by prefill output lengths, and Nested WAIT runs independently for each group. Notably, the number of output lengths, m , minimally impacts performance, which depends primarily on total arrival rates, as shown later. The number of segments can also differ from m , while still retaining near-optimal guarantees under similar conditions.

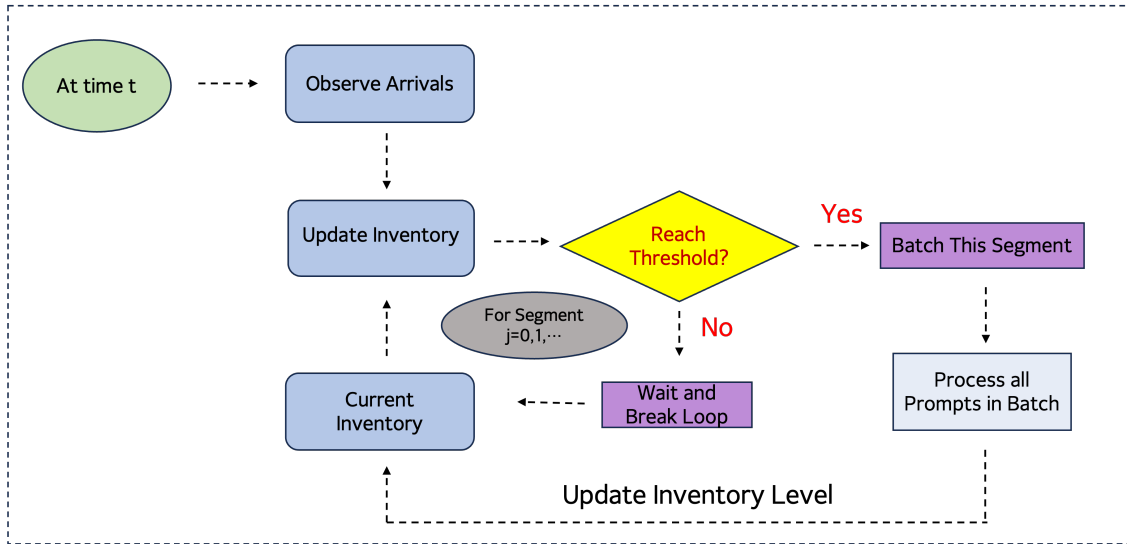


Figure 5: Pipeline of the Nested WAIT Algorithm

To further illustrate how our Nested WAIT works, consider a simplified scenario with two prompt types,

both having the same initial input length l , but differing in decode length, with $l'_1 < l'_2$. While prompt types are indistinguishable at arrival, they become identifiable after one decode iteration. To accommodate this, the Nested WAIT algorithm partitions processing into two segments. **Segment 1** includes all prompts at stages $0, 1, \dots, l'_1$, and thus contains both types of prompts immediately after arrival. **Segment 2**, in contrast, includes only type-2 prompts at later stages $l'_1 + 1, \dots, l'_2$, once type-1 prompts have completed. The effective arrival rate for Segment 1 is $\lambda_1 + \lambda_2$, while Segment 2 receives prompts at a reduced rate λ_2 , delayed by one decode step. Figure ?? visualizes the algorithm in action. At time $t = 0$, prompts $P1$ and $P2$ arrive but are insufficient to trigger processing, as Segment 1 has fewer than 3 prompts. At $t = 1$, $P3$ arrives, satisfying the Segment 1 threshold, and the batch is processed. Since these prompts have just begun decoding, none qualify for Segment 2. At $t = 5$, a new wave of prompts ($P4, P5$) arrives, again below the threshold. By $t = 7$, the threshold is met with $P6, P7$, and processing proceeds. Now, $P3$, having reached a deeper decode stage, transitions to Segment 2. However, since the Segment 2 threshold is still unmet, only Segment 1 is processed. Finally, at $t = 14$, enough prompts have advanced into Segment 2 ($P3, P5, P6$, etc.) to satisfy both segment thresholds. At this point, the Nested WAIT algorithm jointly schedules both segments, maximizing GPU efficiency by batching according to type-specific readiness. This example illustrates how the algorithm defers processing to ensure type separation and stage-aligned batching, achieving high utilization while respecting decode-stage heterogeneity and unknown output.

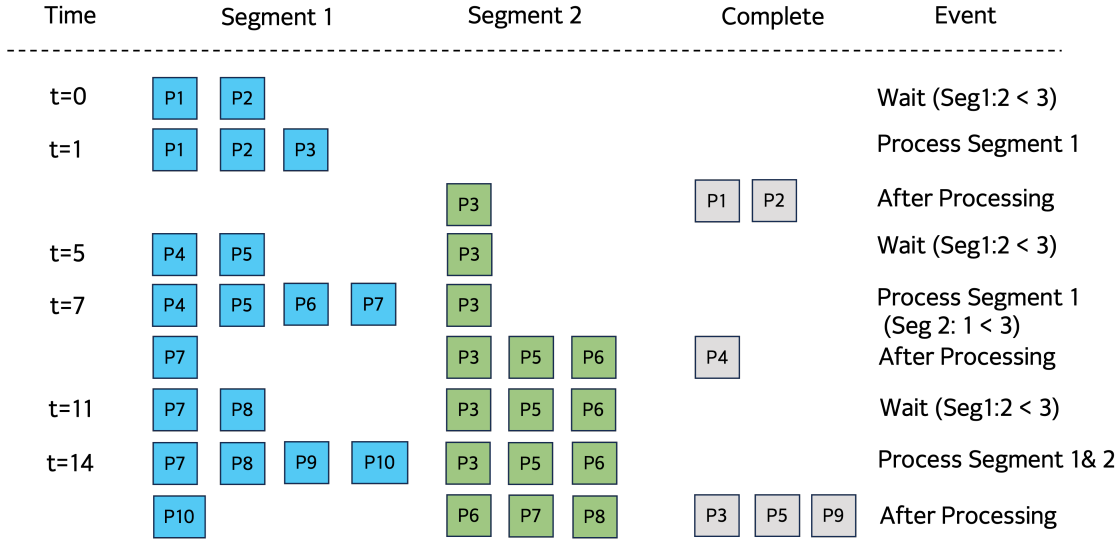


Figure 6: Example of the Nested WAIT Algorithm

Key Insight: On-the-fly Classification. A crucial feature of Nested WAIT is that it achieves *implicit type classification without prediction*. Unlike approaches that attempt to predict output lengths at arrival, our algorithm lets prompt types reveal themselves naturally as processing progresses:

- **Short prompts complete early:** Type-1 prompts (with shortest output l'_1) complete within Segment 1, freeing their memory immediately.
- **Long prompts continue:** Prompts that do not complete at the segment boundary are automatically classified as longer types and advance to the next segment.
- **No blocking:** Short prompts are never delayed by long prompts waiting in later segments, as each segment operates with its own threshold.

This on-the-fly classification is key to avoiding the inefficiency of worst-case assumptions. For instance, with output lengths ranging from 1 to 10,000, assuming all prompts require 10,000 tokens would waste 99.99% of memory for short prompts. Nested WAIT instead processes short prompts quickly while gracefully handling long ones, achieving near-optimal memory utilization across the entire output length distribution.

Algorithm 2 Nested WAIT: Nested Waiting for Accumulated Inference Threshold

Input: Memory capacity C , arrival rates λ_j for $j \in [m]$, thresholds n_i for segments $i \in [m]$

Input: Output lengths $l'_1 < l'_2 < \dots < l'_m$ for prompt types $j \in [m]$

```
1: Initialize segment inventories  $n_{i,k} \leftarrow 0$  for all  $i \in [m], k \in \{\sum_{j=1}^{i-1} l'_j, \dots, \sum_{j=1}^i l'_j - 1\}$ 
2: Initialize event queue
3: Set current time  $t \leftarrow 0$ 
4: while True do
5:   Wait for the next event (arrival or batch completion)
6:   if event is an arrival then
7:     Update inventory:  $n_{1,0} \leftarrow n_{1,0} + 1$   $\triangleright$  New prompts start in segment 1, stage 0
8:   else if event is a batch completion for segment  $i$  then
9:     Update inventory: For each stage  $k$  in segment  $i$ ,  $n_{i,k} \leftarrow n_{i,k} - \min\{n_i, n_{i,k}\}$ 
10:    Advance prompts: Move  $\min\{n_i, n_{i,k}\}$  prompts from stage  $k$  to  $k+1$  within segment  $i$ 
11:    if prompts reach stage  $l'_i + 1$  then
12:      if  $i < m$  then
13:        Move incomplete prompts to segment  $i+1$ , stage  $l'_i + 1$   $\triangleright$  Prompts are of types  $\geq i+1$ 
14:      else
15:        Clear KV caches for completed prompts
16:      end if
17:    end if
18:  end if
19:  Find the largest  $i$  such that  $n_{i', \sum_{j=1}^{i'-1} l'_j} \geq n_{i'}$  for all  $i' = 1$  to  $i$ 
20:  if such an  $i$  exists then
21:    Form batch  $B$  from segments  $i' \leq i$  by selecting  $\min\{n_{i'}, n_{i',k}\}$  prompts at each stage  $k$ 
22:    Process batch  $B$ , keep prompts outside the batch waiting and do not delete their KV caches
23:  end if
24: end while
```

5.2 Main Theorem

We evaluate Nested WAIT's performance in the asymptotic regime (Section ??). Unknown prompt types increase complexity, as variable output lengths affect memory and processing times. The primary metric is average throughput, $\mathbf{Throughput}^{(\zeta, \pi)}$, where ζ is the scaling parameter and π is the Nested WAIT policy. We also provide theoretical results for average latency $\mathbf{Latency}^{(\zeta, \pi)}$ and time-to-first-token $\mathbf{TTFT}^{(\zeta, \pi)}$.

To achieve an asymptotic performance gap of $o(1)$, memory M must exceed the equilibrium memory in Equation (??), in order to mitigate overflow risks due to initially unknown output lengths. The following proposition shows that the worst-case throughput gap between any non-predictive policy unaware of arrival types and the fluid throughput (Equation (??)) is at least $\Omega(1)$.

Proposition 5 *When memory capacity C equals the equilibrium memory M^* , there exists an instance where any online non-predictive policy π , unaware of prompt types upon arrival, incurs a throughput gap $\mathbf{Throughput}^* - \mathbb{E}[\mathbf{Throughput}^{(\zeta, \pi)}] = \Omega(1)$.*

Intuitively, with $C = M^*$, ignorance of output lengths risks under-utilization or overflow after each iteration. Overflow forces restarts, reducing efficiency. Both scenarios yield a throughput gap of $\Omega(1/(\zeta T))$, accumulating to $\Omega(1)$ over time.

Similar to Equation (??), we define $\Delta T_{[1, \dots, m]}(n_1, \dots, n_m)$ as the time cost per iteration with exactly n_i prompts per stage in segment i , for all $i \in [1, \dots, m]$. Thresholds must satisfy:

$$\begin{aligned} \Delta T_{[1, \dots, m]}(n_1, \dots, n_m) &< \frac{n_1}{\sum_{j=1}^m \lambda_j}, \\ \frac{n_{i+1}}{n_i} &> p_i, \forall i \in \{1, \dots, m-1\}, \end{aligned} \tag{12}$$

where $p_i = (\sum_{j=i+1}^m \lambda_j) / (\sum_{j=i}^m \lambda_j) < 1$. Additionally, we define the memory usage when running iteration

with exactly n_i prompts per stage in segment i , for all $i \in [1, \dots, m]$ as:

$$M^\pi = \sum_{j=1}^m n_j \left(l + \frac{1}{2} L'_j \right) \Delta l'_j, \quad (13)$$

$$\text{where } L'_j = \sum_{k=1}^j l'_k, \quad \Delta l'_j = l'_j - l'_{j-1}, \quad l'_0 = 0.$$

We define that the total number of iterations over time horizon $[0, T]$ as S . Therefore, we have $T \geq \sum_{s=1}^S \Delta T_{s\text{-th Batch}} \geq S \cdot \min \Delta T_{\text{Batch}}$, where $\Delta T_{s\text{-th Batch}}$ denotes the time cost of the s -th iteration and $\Delta T_{\min} = \min_s \{\Delta T_{\text{Batch}}\}$. The following theoretical result demonstrates Nested WAIT's asymptotic optimality in the asymptotic regime with unknown prompt types, given thresholds $\pi = [n_1, \dots, n_m]$ satisfying Equation (??).

Theorem 2 Assume thresholds $\pi = [n_1, \dots, n_m]$ satisfy Equation (??), and memory M satisfies:

$$\begin{aligned} & M^{(\zeta, \pi)} \\ & \geq M^\pi + \sum_{j=2}^m (l + l'_{j-1}) \left(n_j + \theta_j^{-1} \ln \left(\frac{m\zeta S}{\delta} \right) \right) \\ & = O \left(2M^\pi + \sum_{j=1}^m (l + l'_{j-1}) \theta_j^{-1} \ln \left(\frac{m\zeta S}{\delta} \right) \right), \end{aligned} \quad (14)$$

where θ_j is with lower bound $8(n_j - n_{j-1}p_j)/n_j$. Then:

$$\begin{aligned} & \text{Throughput}^* - \mathbb{E}[\text{Throughput}^{(\zeta, \pi)}] = O((\zeta T)^{-1}), \\ & \mathbb{E}[\text{Latency}^{(\zeta, \pi)}], \mathbb{E}[\text{TFTT}^{(\zeta, \pi)}] = O(1). \end{aligned}$$

Moreover, memory is not exceeded with probability at least $1 - \delta$ during time horizon $[0, T]$.

The proof relies on constructing a coupling process that dominates the true dynamics with more tractable transition kernel. We can then adopt analytic tools for Lindley process (see e.g. ?) and Doob's maximal inequality for submartingale to establish the performance gap.

Understanding the Safety Buffer. The memory requirement in Equation (??) consists of two components with clear operational interpretations:

1. **Base memory** M^π : This is the equilibrium memory from the fluid model—the memory needed when exactly n_j prompts occupy each stage in segment j . This represents the “steady-state” memory consumption.
2. **Safety buffer** $\sum_{j=2}^m (l + l'_{j-1}) (n_j + \theta_j^{-1} \ln(m\zeta S/\delta))$: This additional memory hedges against stochastic fluctuations when output lengths are unknown. The key insight is:
 - The term n_j accounts for prompts that may queue at segment boundaries waiting for thresholds to be met.
 - The term $\theta_j^{-1} \ln(m\zeta S/\delta)$ provides high-probability protection against queue buildup, ensuring memory overflow occurs with probability at most δ .
 - The coefficient $(l + l'_{j-1})$ reflects the memory footprint of prompts at the boundary between segments $j - 1$ and j .

Crucially, this safety buffer is *logarithmic* in the time horizon T (via S) and confidence level $1/\delta$, making it a modest overhead compared to the base memory M^π . This is why Proposition ?? shows that *some* extra memory beyond M^* is necessary for unknown output lengths, but our Theorem ?? demonstrates that only a logarithmic amount suffices.

There are three notable observations: (i) The algorithm’s performance exhibits little dependence on the number of prompt types m ; instead, it is primarily determined by the arrival rates and output lengths. For large m (e.g., output lengths $\{1, 2, \dots, m\}$), employing descending thresholds $n_1 > n_2 > \dots > n_m$ and batching up to n_i prompts per stage remains effective when arrival rates are moderate, as indicated by condition (??). (ii) The memory requirement given in (??) is generally dominated by the first term M^π . Further details and numerical support are provided in Section ?? . (iii) As previously discussed, the segment structure can be extended from m segments to an arbitrary number L . Algorithm ?? retains asymptotic optimality, provided similar conditions to those specified in (??) are satisfied.

6 Numerical Experiments

In this section, we present a comprehensive evaluation of our (Nested) WAIT algorithm through numerical experiments. Our analysis focuses on two distinct scenarios: (i) simulated prompts designed to test the algorithm under controlled conditions, and (ii) a real-world dataset sourced from ? to assess its performance in practical settings. All experiments are conducted using Microsoft Vidur to simulate an NVIDIA A100 GPU, mirroring the hardware configuration employed in prior studies (?). We benchmark our WAIT algorithm against vLLM (?) and Sarathi (?), widely recognized inference engines that employ First-Come-First-Serve (FCFS) scheduling. Specifically, vLLM prioritizes new arrivals, while Sarathi prioritizes ongoing prompts, with both enforcing fixed limits on the total number of tokens and prompts per iteration.

6.1 Experiment on WAIT Algorithm

We first evaluate the WAIT algorithm when prompt output lengths are known upon arrival. Simulated prompts are generated with varying input lengths l_j and output lengths l'_j , drawn from distributions that emulate real-world variability. Arrival rates λ_j are adjusted to simulate both low-traffic and high-traffic scenarios, testing the algorithm’s resilience under stress. Specifically, we generate type j prompts via a Poisson process with rate λ_j and test two scenarios:

- (i) **Low demand:** $m = 2$, $(l_1, l_2) = (10, 10)$, $(l'_1, l'_2) = (10, 20)$, $(\lambda_1, \lambda_2) = (1000, 1000)$;
- (ii) **High demand:** $m = 3$, $(l_1, l_2, l_3) = (20, 20, 20)$, $(l'_1, l'_2, l'_3) = (100, 200, 300)$, $(\lambda_1, \lambda_2, \lambda_3) = (6000, 4000, 2000)$.

For practical implementation, we simplify threshold computation in WAIT. Given a batch size limit B , we set the threshold for type j prompts as $n_j = B \cdot \rho_j / (l'_j + 1)$, where $\rho_j = \lambda_j / \sum_{j'=1}^m \lambda_{j'}$, aligning with Algorithm ?? . To ensure a fair comparison, we enforce the same batch size limit across WAIT, Sarathi, and vLLM.

The throughput of WAIT surpasses that of the benchmark algorithms across all tested arrival rates, as illustrated in the left panels of Figure ?? and Figure ?? . This improvement stems from WAIT’s ability to optimize batch formation, minimizing GPU idle time. Latency, shown in the right panels, remains within acceptable limits, highlighting WAIT’s balance between efficiency and responsiveness.

6.2 Experiment on Nested WAIT Algorithm

Next, we evaluate the Nested WAIT algorithm, designed for scenarios where prompt output lengths are unknown. We test it on both synthetic and real-world datasets, using the same batch size limits across all algorithms and setting thresholds proportional to arrival rates in each segment for practical implementation.

We begin with a synthetic dataset where $m = 4$, prefill lengths are fixed at 10 tokens, decode lengths are (20, 40, 80, 160), and arrival rates follow a 20 : 40 : 80 : 160 ratio. We set thresholds (n_1, n_2, n_3, n_4) with the ratio 15 : 14 : 12 : 8. Figure ?? shows improved throughput and latency over benchmarks.

For real-world testing, we use the dataset from ?, which contains over 210,000 prompts from Vicuna and Chatbot Arena. We treat questions as prompts and answers as decoded tokens. Figure ?? shows the distribution of prefill and decode lengths. We randomly sample 50,000 prompts and simulate arrivals via a Poisson process.

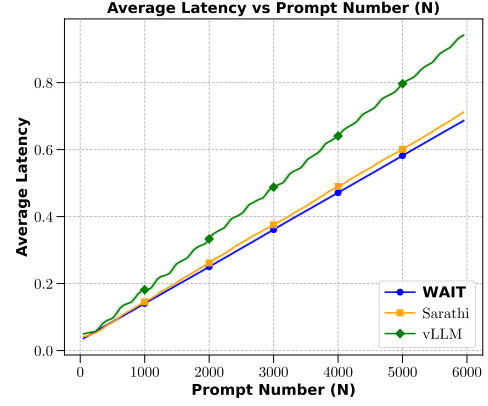
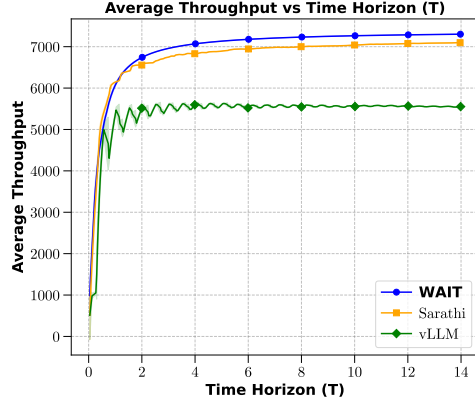


Figure 7: Average throughput and latency across algorithms on the synthetic dataset with low demand.

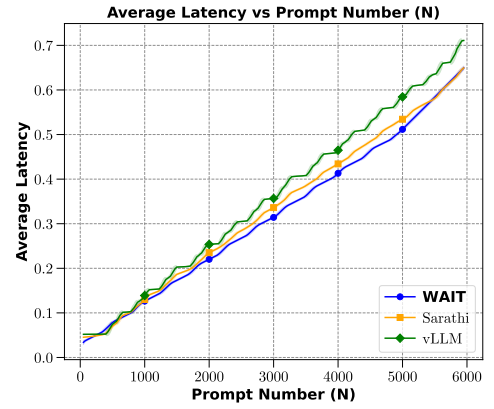
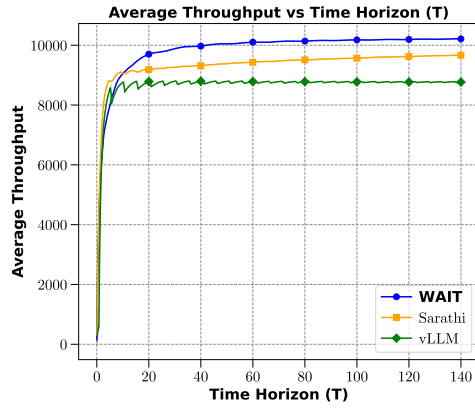


Figure 8: Average throughput and latency across algorithms on the synthetic dataset with high demand.

To implement Nested WAIT, we group decode lengths (1 to 500 tokens) into 10 bins of 50 tokens each (e.g., [1-50], [51-100], etc). For each bin, we compute the mean prefill length (approximately 60 tokens, as shown in Figure ??) and arrival rates proportional to 23 : 11 : 8 : 7 : 6 : 4 : 3 : 2 : 1 : 1. We set $m = 10$ and thresholds $(n_1, \dots, n_{10})$ with the ratio 66 : 43 : 32 : 24 : 17 : 11 : 7 : 4 : 2 : 1, following (??). Prompts in the k -th decode stage are assigned to segment $\lceil k/50 \rceil$.

We first test a synthetic workload with prefill fixed at 60 tokens and decode lengths clustered into 10 types (50, 100, ..., 500 tokens), with arrival rates matching the real dataset's proportions. We evaluate Nested WAIT with queries per second (QPS) of 55 and 550, shown in Figure ?? and Figure ??, respectively. Nested WAIT outperforms benchmarks in both throughput and latency.

Finally, we test the real dataset with original prefill and decode lengths, using the same thresholds. Figure ?? shows that Nested WAIT achieves higher throughput with a modest increase in latency (approximately 0.5 seconds per prompt), enhancing system efficiency while maintaining responsiveness. The designed thresholds are on time-varying arrival rate but with static threshold, which leads to a little bit more latency.

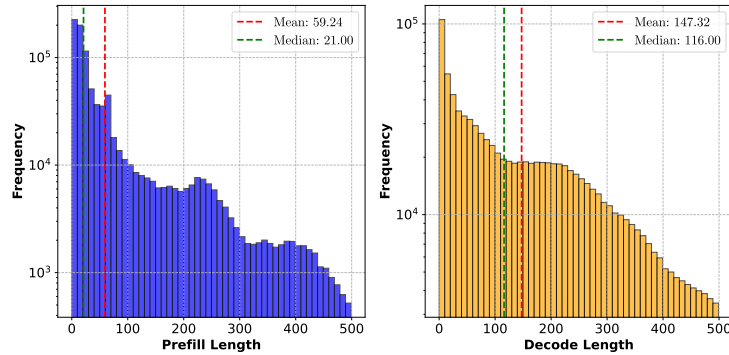


Figure 9: Distribution of prefill and decode lengths in the real dataset.

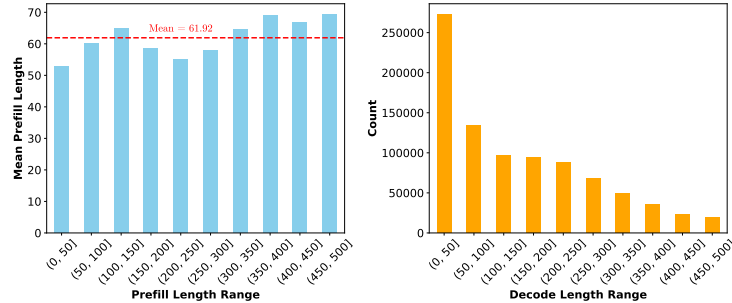


Figure 10: Mean prefill lengths and prompt counts per decode length bin in the real dataset.

7 Extensions

In this section, we present two extensions to our generalize our algorithms and theoretic analysis. First, we investigate the adaptation of Algorithm ?? to accommodate time-varying arrival rates. Second, we propose a generalized segment design for Algorithm ??, extending beyond the existing m -segment parameterization. We establish that these extensions preserve comparable theoretical guarantees as in Theorem ??.

7.1 Time-Varying Arrival Rates

In this part, we analyze the asymptotic optimality of the Nested WAIT algorithm under time-varying arrival rates, denoted λ_j^t for prompt type $j \in [m]$ at time $t \in [0, T]$. We assume that λ_j^t are continuous functions of

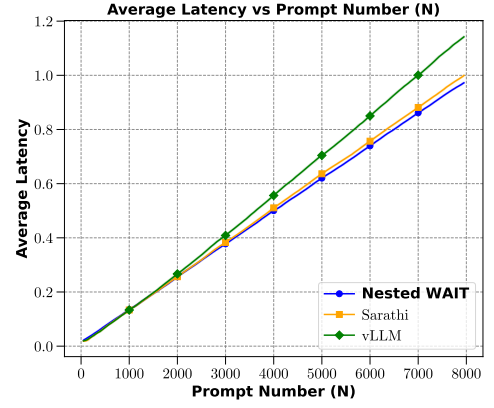
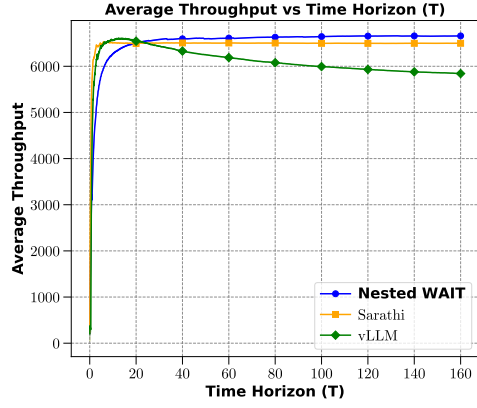


Figure 11: Average throughput and latency across algorithms on the synthetic dataset.

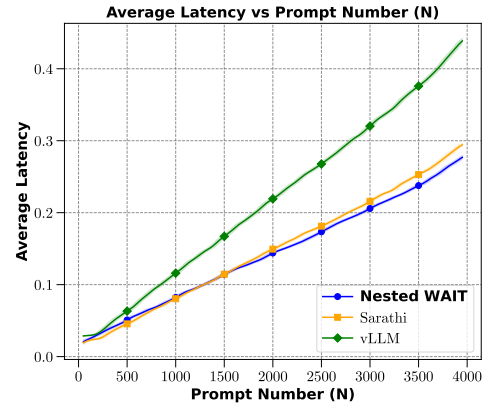
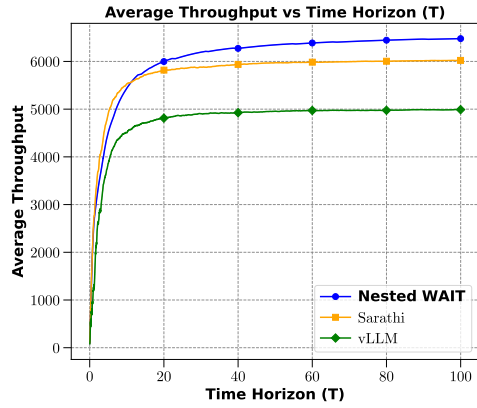


Figure 12: Average throughput and latency across algorithms on the real dataset with clustered output lengths (low arrival rates).

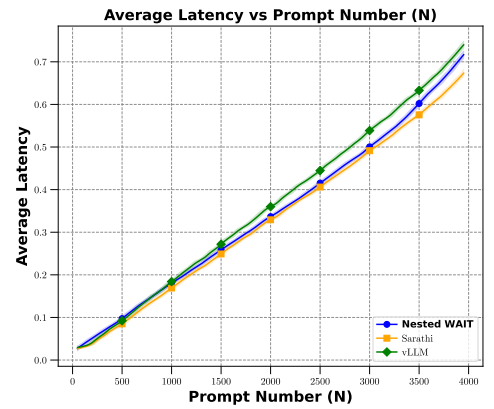
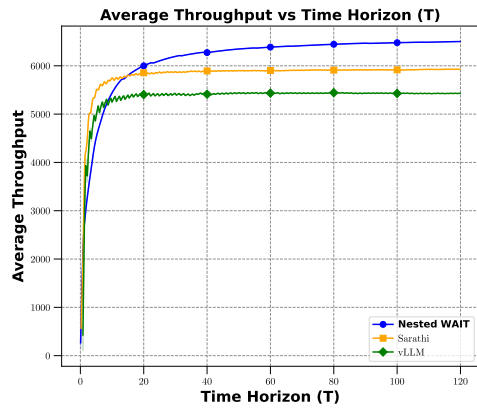


Figure 13: Average throughput and latency across algorithms on the real dataset with clustered output lengths (high arrival rates).

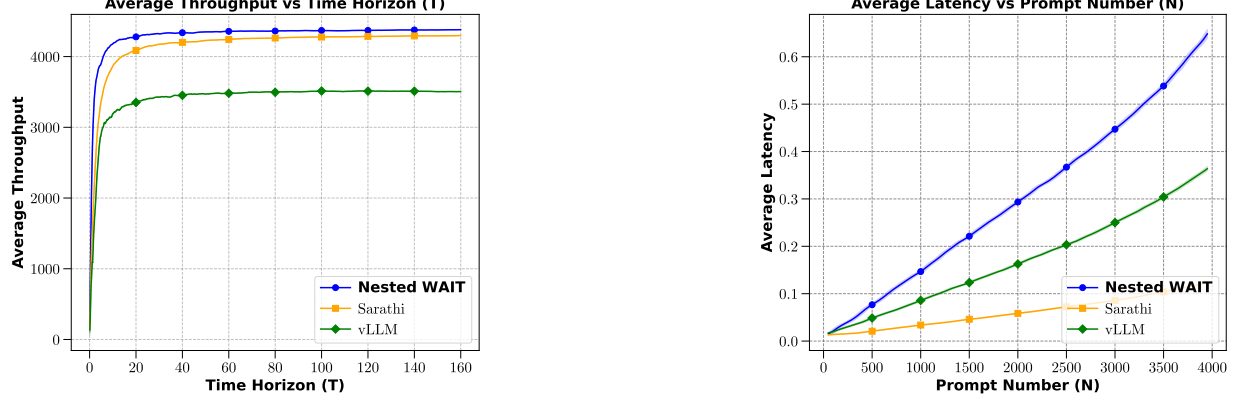


Figure 14: Average throughput and latency across algorithms on the real dataset with actual output lengths.

time, uniformly bounded by $\lambda_{\max} < \infty$. Define the accumulated arrivals of type j over the interval $[t_1, t_2]$ as $\lambda_j[t_1, t_2] := \int_{t_1}^{t_2} \lambda_j^t dt$. Recall from Equation (??) that $\Delta T_{[1,2,\dots,m]}(n_1, \dots, n_m)$ represents the per-iteration time cost when processing exactly n_i prompts at each stage in segment i , for all $i \in [m]$, as (??) (n_i is the threshold defined in Algorithm ??).

For the Nested WAIT policy defined by $\pi = [n_1, \dots, n_m]$, we consider the following conditions for all $t \in [0, T]$, $0 < \Delta t \leq \Delta T_{[1,2,\dots,m]}(n_1, \dots, n_m)$ and $i = 1, 2, \dots, m-1$, analogous to (??):

$$\begin{aligned} & \sum_{j=1}^m \lambda_j[t, t + \Delta t] \\ & \leq \sum_{j=1}^m \lambda_j[t, t + \Delta T_{[1,2,\dots,m]}(n_1, \dots, n_m)] < n_1, \\ & n_{i+1}/n_i > p_i[t, t + \Delta t], \end{aligned} \tag{15}$$

where $p_i[t, t + \Delta t] = (\sum_{j=i+1}^m \lambda_j[t, t + \Delta t]) / (\sum_{j=i}^m \lambda_j[t, t + \Delta t])$.

We define that the total number of iterations over time horizon $[0, T]$ as S . Therefore, we have $T \geq \sum_{s=1}^S \Delta T_{s\text{-th Batch}} \geq S \cdot \min \Delta T_{\text{Batch}}$, where $\Delta T_{s\text{-th Batch}}$ denotes the time cost of the s -th iteration and $\Delta T_{\min} = \min_s \{\Delta T_{\text{Batch}}\}$. We now present the following result that extends Theorem ??.

Theorem 3 Suppose the thresholds $\pi = [n_1, \dots, n_m]$ satisfy (??), and the memory $M^{(\zeta, \pi)}$ fulfills:

$$\begin{aligned} M^{(\zeta, \pi)} & \geq M^\pi + \sum_{j=2}^m (l + l'_{j-1}) \left(n_j + \theta_j^{-1} \ln \left(\frac{m\zeta S}{\delta} \right) \right) \\ & = O \left(2M^\pi + \sum_{j=1}^m \theta_j^{-1} (l + l'_{j-1}) \ln \left(\frac{m\zeta S}{\delta} \right) \right), \end{aligned}$$

where the memory usage M^π is defined as (??) and θ_j is given in (??) with the lower bound in Equation (??). Then, the following asymptotic bounds hold:

$$\begin{aligned} \text{Throughput}^* - \mathbb{E} [\text{Throughput}^{(\zeta, \pi)}] & = O((\zeta T)^{-1}), \\ \mathbb{E} [\text{Latency}^{(\zeta, \pi)}], \mathbb{E} [\text{TFT}^{(\zeta, \pi)}] & = O(1). \end{aligned}$$

Additionally, the memory usage remains below $M^{\zeta, \pi}$ with probability at least $1 - \delta$ over the time horizon $[0, T]$.

The proof constructs a coupled dominant stochastic process, extending the approach of Theorem ?? with adjustments for time-varying arrival rates. A detailed proof is provided in Appendix ??.

7.2 Segment Design

In this subsection, we generalize the segment design in Algorithm ?. We assume constant arrival rates, though the approach can be extended to time-varying rates as discussed earlier. Consider m prompt types with decode lengths $l'_1 < \dots < l'_m$ and arrival rates $\lambda_1, \lambda_2, \dots, \lambda_m$. Our objective is to merge these into L segments, where $L \leq m$. For notational convenience, let $\Delta L := m/L$. We partition the m types into L segments as follows: types 1 to ΔL , types $\Delta L + 1$ to $2\Delta L$, ..., types $(L-1)\Delta L + 1$ to m .

For each segment $i \in \{1, \dots, L\}$, the total arrival rate is defined as $\lambda'_i = \sum_{(i-1)\Delta L + 1 \leq j \leq i\Delta L} \lambda_j$. Additionally, we define $\lambda'(i_1 \rightarrow i_2) := \sum_{j=i_1}^{i_2} \lambda'_j$ for $1 \leq i_1 \leq i_2 \leq L$.

Given a policy $\pi = [n_1, \dots, n_L]$ with threshold n_i for the i -th segment in Algorithm ?, we formulate a linear system analogous to Equation (??):

$$\begin{aligned} \Delta T_{[1:L]}(n_{1:L}) &\leq \frac{n_1}{\sum_{j=1}^L \lambda'_j}, \\ \frac{n_{i+1}}{n_i} &> p_i, \quad \forall i < L, \end{aligned} \tag{16}$$

where $p_i = \frac{\lambda'(i+1 \rightarrow L)}{\lambda'(i \rightarrow L)}$, $\Delta T_{[1,\dots,L]}(n_1, \dots, n_L) = d_0 + d_1 M^\pi(n_1, \dots, n_L)$, and the memory M^π running with n_i at each stage in segment i is given by:

$$M^\pi(n_1, \dots, n_L) = \sum_{i=1}^L n_i \left(l + \frac{i}{2} \Delta L \right) \Delta L.$$

We define that the total number of iterations over time horizon $[0, T]$ as S . Therefore, we have $T \geq \sum_{s=1}^S \Delta T_{s\text{-th Batch}} \geq S \cdot \min \Delta T_{\text{Batch}}$, where $\Delta T_{s\text{-th Batch}}$ denotes the time cost of the s -th iteration and $\Delta T_{\min} = \min_s \{\Delta T_{\text{Batch}}\}$. We demonstrate that the Nested WAIT algorithm (Algorithm ?) with L segments and thresholds $[n_1, \dots, n_L]$ satisfying Equation (??) achieves asymptotic optimality in the asymptotic regime as outlined in Section ?.

Theorem 4 *For any fixed number of segments L and thresholds $\pi = [n_1, \dots, n_L]$ satisfying Equation (??), if the memory capacity $M^{(\zeta, \pi)}$ satisfies:*

$$\begin{aligned} M^{(\zeta, \pi)} &\geq M^\pi + \sum_{j=2}^L (l + l'_{j-1}) n_j \\ &\quad + \sum_{j=2}^L (l + l'_{j-1}) \theta_j^{-1} \ln \left(\frac{m\zeta S}{\delta} \right) \\ &= O \left(2M^\pi + \sum_{j=1}^L (l + l'_{j-1}) \theta_j^{-1} \ln \left(\frac{m\zeta S}{\delta} \right) \right), \end{aligned} \tag{17}$$

where θ_j is given by Equation (??) with lower bound $8(n_j - n_{j-1}p_j)/n_j$, then the performance of Nested WAIT is guaranteed by:

$$\begin{aligned} \text{Throughput}^* - \mathbb{E} \left[\text{Throughput}^{(\zeta, \pi)} \right] &= O((\zeta T)^{-1}), \\ \mathbb{E} \left[\text{Latency}^{(\zeta, \pi)} \right], \mathbb{E} \left[\text{TFT}^{(\zeta, \pi)} \right] &= O(1). \end{aligned}$$

Moreover, memory is not exceeded with probability at least $1 - \delta$ over the total batch counts $s \in \{1, \dots, S\}$ as well as the time horizon $t \in [0, T]$.

The proof follows a similar structure to that of Theorem ??, substituting the original arrival rates $(\lambda_1, \dots, \lambda_m)$ with the clustered rates $(\lambda'_1, \dots, \lambda'_L)$ as in Appendix ?. We omit here for simplicity.

To show how the memory requirements in (??) varies with different model parameters, we utilize the dataset from ? (Section ??), with $m = 500$ prompt types, decode lengths ranging from 1 to 500, and a fixed prefill length of 62. The distribution of prompt types is shown in Figure ??, where arrival rates are set proportional to normalized frequencies.

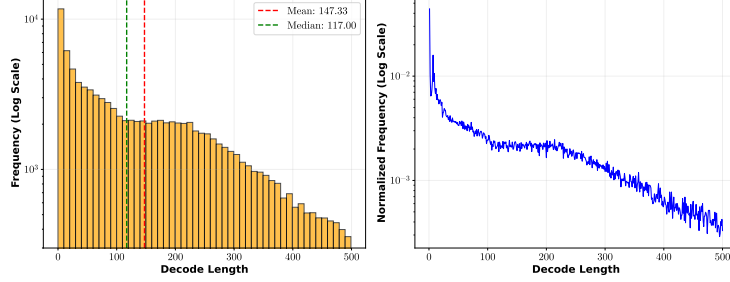


Figure 15: Arrival rate construction from real data, with rates proportional to normalized frequencies.

We analyze the three components of the memory bound in Equation (??): (1) the peak batch memory usage $\sum_{j=1}^L n_j (l + \frac{j}{2}\Delta) \Delta$, (2) the queue length bound $\sum_{j=2}^L n_j (l + l'_{j-1})$, and (3) the high-probability queue length bound $\sum_{j=2}^L \theta_j^{-1} (l + l'_{j-1}) \ln \left(\frac{(L-1)(T+1)}{\delta} \right)$.

Simulations under varying total arrival rates, time horizons T , and confidence levels δ are presented in Figures ??, ??, and ?. These figures depict the memory usage proportions of the three terms across different segment numbers L , revealing that the overall memory demand is primarily driven by the first term (fluid equilibrium memory) and follows a U-shaped trend with respect to L , suggesting that moderate segment number can reduce waste of memory usage. Additionally, Nested WAIT effectively manages unknown prompt types with only a marginal increase in memory beyond the fluid equilibrium requirement. In practice, a moderate number of segments, such as $L = 10$ as used in Section ??, is sufficient.

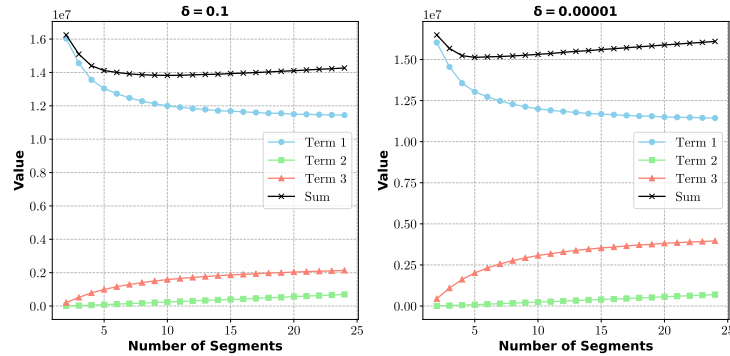


Figure 16: Memory usage proportions under low arrival rate (total rate = 50, $T = 200$). Left: $\delta = 0.1$; Right: $\delta = 10^{-5}$.

8 Conclusion, Limitations, and Future Directions

In this paper, we introduce the (Nested) WAIT algorithm, which offers theoretical guarantees for online scheduling in large language model (LLM) inference problems, even when the output length is unknown upon arrival. Despite its theoretical properties and performance under idealized conditions, determining the optimal parameters for the (Nested) WAIT algorithm remains an unresolved challenge, as does the

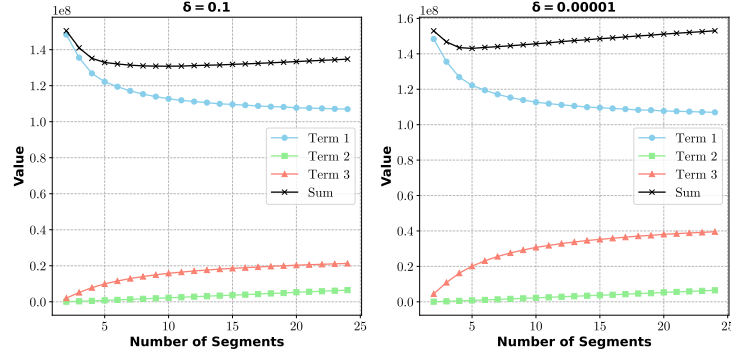


Figure 17: Memory usage proportions under high arrival rate (total rate = 500, $T = 200$). Left: $\delta = 0.1$; Right: $\delta = 10^{-5}$.

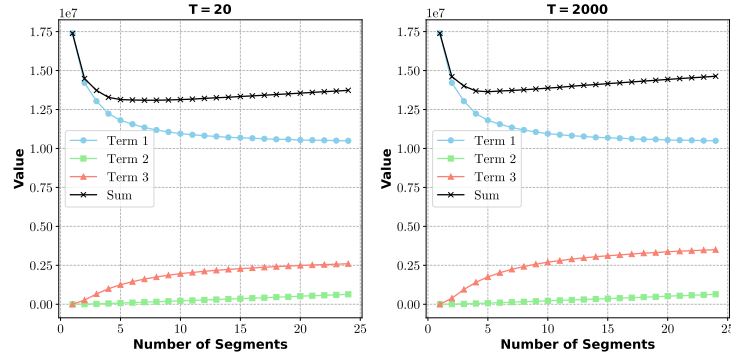


Figure 18: Memory usage proportions with varying time horizons (total rate = 50, $\delta = 0.001$). Left: $T = 20$; Right: $T = 2000$.

associated theoretical analysis. Furthermore, extending the algorithm to multi-GPU scenarios—prevalent in real-world applications—introduces additional complexities. Additionally, since more and more system-level optimization methods for LLM inference have been proposed, how to build on analytical frameworks for these newly prevalent models becomes necessary. In the future, we plan to strategically invest in research across the following directions.

8.1 More Volatile Arrivals

In practical settings, the distribution of incoming prompts fluctuates over time, violating the stationarity assumption underlying our model. While our algorithm can be directly adapted to scenarios with moderate variations in arrival rates—provided they remain within system capacity—by re-solving the fluid equilibrium, a thorough analysis of its performance in more volatile environments, such as during demand surges, remains an open question. Investigating how the (Nested) WAIT algorithm behaves under such conditions is an important avenue for future research.

8.2 Extension to Multi-GPU Systems

Our current research is limited to single-GPU systems, capable of supporting LLMs such as Llama-13B. Extending the algorithm to multi-GPU environments introduces additional considerations, including communication costs, parallelization strategies, and hardware constraints. For example, while single-GPU deployment is relatively straightforward, multi-GPU setups necessitate careful management of data parallelism, tensor parallelism, and pipeline parallelism. Moreover, communication overhead becomes a critical factor,

particularly since many large-scale models require multiple GPUs for deployment. Developing a theoretical framework that incorporates these elements poses a significant challenge, which we plan to investigate further.

8.3 Analytical Framework for Up-to-Date LLM Inference Models

In recent years, system-level designs for LLM inference have significantly influenced the field. For instance, the multi-head latent attention (MLA) mechanism introduced by ? has achieved a 90% reduction in memory cost through an innovative compression technique. Integrating these state-of-the-art system-level optimizations into a theoretical framework presents a promising direction for future research.

Endnotes

We sincerely thank Dr. Zijie Zhou and Prof. Jing Dong for their valuable suggestions and insightful comments, which significantly helped improve the quality of this work.

A Proof of Theorem ??

First we consider the single-type case, and prove some useful lemmas.

A.0.1 Single-Type

First, consider the single-type scenario. For the booking limit policy, it can be inferred from the recursion that the waiting time is necessarily spent on awaiting the accumulation of sufficient prompts for prefill.

Lemma 1 *The number of incoming prompts X^t at each time step n follows a Poisson distribution with parameter λ . The service processes up λ prompts at each time step when there are at least λ prompts available. Let W^t represent the number of prompts in the queue at time t , starting with $W^0 = 0$. The state transition rule is given by*

$$W^{t+1} = W^t + X^t - \lambda \cdot \mathbf{1}\{W^t + X^t \geq \lambda\}$$

Define the total number of stuck time steps as T_{stuck} , which is also the total number of time steps of $W^t + X^t < \lambda$, and the following relationship holds

$$\mathbb{E}[W^T] = \lambda \cdot \mathbb{E}[T_{stuck}].$$

Detailed proof is in Appendix ??. In order to estimate the expected upper bound, we need to construct a coupled process to dominate W^t .

Lemma 2 *Define a new process $\{\tilde{W}^t\}$ as a coupled version of $\{W^t\}$ as follows:*

$$\tilde{W}^0 = 2\lambda, \quad \tilde{W}^{t+1} = \max\{2\lambda, \tilde{W}^t + X^t - \lambda\}.$$

Then, we have $\tilde{W}^t \geq W^t + \lambda, \forall t \in \mathbb{N}$.

Detailed proof is in Appendix ??. Next, by Proposition 6.3 in ?, we have that

Lemma 3

$$\tilde{W}^T = 2\lambda + \max(S^T, S^T - S^1, \dots, S^T - S^{T-1}, 0), \quad S^k = \sum_{r=1}^k (X^r - \lambda)$$

Let $\xi^k = X^k - \lambda$, and notice that

$$\max(S^T, S^T - S^1, \dots, S^T - S^{T-1}, 0) = \max_{1 \leq k \leq T} \left(\sum_{r=k}^T \xi^r \right)^+ \stackrel{d}{=} \max_{1 \leq s \leq T} \left(\sum_{r=1}^s \eta^r \right)^+, \quad \eta^r \stackrel{d}{=} X^1 - \lambda$$

So we have that $\tilde{W}^T \stackrel{d}{=} 2\lambda + \max_{1 \leq s \leq T} (\sum_{r=1}^s \eta^r)^+$.

Since $\tilde{W}^T \geq \lambda^T$, we have $\mathbb{E}[W^T] \leq \mathbb{E}[\tilde{W}^T] - \lambda = \mathbb{E}[\max_{1 \leq s \leq T} (\sum_{r=1}^s \eta^r)^+] + \lambda$. Let $Y^T = \max_{1 \leq s \leq T} (\sum_{r=1}^s \eta^r)^+$, consider the expectation $\mathbb{E}[Y^T]$. By Lemma 2 in ?, we get that

Lemma 4

$$\mathbb{E}[Y^T] = \mathbb{E} \left[\max_{1 \leq s \leq T} \left(\sum_{r=1}^s \eta^r \right)^+ \right] = \mathbb{E} \left[\max_{1 \leq k \leq T} (S^k)^+ \right] = \sum_{k=1}^T \frac{1}{k} \mathbb{E}[(S^k)^+]$$

Then we consider the $\mathbb{E}[(S^k)^+]$, and just use Cauchy's inequality

$$\mathbb{E}[(S^k)^+] = \mathbb{E}[\max\{S^k, 0\}] \leq \mathbb{E}[|S^k|] \leq \sqrt{\mathbb{E}[(S^k)^2]} = \sigma\sqrt{k} = \sqrt{\lambda k} \Rightarrow \mathbb{E}[Y^T] \leq \sqrt{\lambda} \sum_{k=1}^T \frac{1}{\sqrt{k}} \sim \sqrt{\lambda T}$$

So we have that

Lemma 5

$$\mathbb{E}[W^T] - \lambda \leq \mathbb{E}[Y^T] \leq \sqrt{\lambda} \sum_{k=1}^T \frac{1}{\sqrt{k}} \sim \sqrt{\lambda T}, \quad T \rightarrow \infty$$

Then consider the average throughput. In the deterministic scenario, the average throughput **Throughput**^{*} = λ . Thus, in the stochastic case, the expected gap between the average throughput **Throughput** and **Throughput**^{*} is given by $\frac{1}{T} \mathbb{E}[W^T]$.

In the **Asymptotic Regime**,

$$\frac{1}{T^\zeta} \mathbb{E}[W^{T,\zeta}] = \frac{\lambda^\zeta}{T^\zeta} + \frac{(\lambda T)^{\zeta/2}}{T^\zeta} = \left(\frac{\lambda}{T}\right)^\zeta + \lambda^{\zeta/2} T^{-\zeta/2} \rightarrow 0 \quad \text{as } \zeta \rightarrow \infty$$

A.0.2 Multiple-Type

When the memory constraint $C \geq M^*$, consider the WAIT policy π , which is defined by the $[n_1, \dots, n_m]$, where n_j represents the threshold of the number of prompts of type j in each stage.

Solving the linear system:

$$d_0 + d_1 \sum_{j=1}^m n_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right) \leq \frac{n_j}{\lambda_j}, \quad M^\pi = \sum_{j=1}^m n_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right) \geq M^*$$

we obtain the policy class Π , each $\pi = [n_1, \dots, n_m] \in \Pi$ with corresponding memory capacity $M^\pi \geq M^*$.

$\forall \pi \in \Pi$, when prompts of all m types are in the same batch, $\text{Arrival}_j \leq \text{Completion}_j$. Indeed, when type j is in the batch, no matter what the other types are in the batch, we have that $\text{Arrival}_j \leq \text{Completion}_j$.

Specifically, when $M^\pi = M$, the policy is $\pi = [\frac{n_1^*}{l'_1+1}, \dots, \frac{n_m^*}{l'_m+1}]$.

Now we define some notations. Let s denote the batch index, J^s denote the types contained in the s -th batch, $s(J^s)$ denote the time at the start of the inference of the s -th batch and $e(J^s)$ denote the time at the end of the inference of the s -th batch. Note that $s(J^{s+1}) - e(J^s)$ denotes the waiting time before $s+1$ -th batch, this waiting condition only occurs when all m types do not satisfy the threshold.

At $s+1$ -th batch, the state transition rule is given by

$$W_{(j)}^{e(J^{s+1})} = W_{(j)}^{e(J^s)} + X_{(j)}^{s(J^{s+1})-e(J^s)} + Y_{(j)}^{e(J^{s+1})-s(J^{s+1})} - n_j \cdot \mathbf{1}\{j \in J^{s+1}\}, \quad \forall j = 1, \dots, m$$

where $X_{(j)}^{s(J^{s+1})-e(J^s)}$ and $Y_{(j)}^{e(J^{s+1})-s(J^{s+1})}$ are independent with

$$X_{(j)}^{s(J^{s+1})-e(J^s)} \sim P(\lambda_j \cdot (s(J^{s+1}) - e(J^s))), \quad Y_{(j)}^{e(J^{s+1})-s(J^{s+1})} \sim P(\lambda_j \cdot (e(J^{s+1}) - s(J^{s+1})))$$

Since $s(J^{s+1}) - e(J^s) > 0$ if and only if when all m types do not satisfy the threshold, so $\forall t \in [e(J^s), s(J^{s+1})]$, $W_{(j)}^{e(J^s)} + X_{(j)}^t \leq n_j, \forall j \in \{1, \dots, m\}$. We will use this property to construct a coupled process to dominate this.

Lemma 6 Define a coupled process $\tilde{W}_{(j)}^t$ with continuous time:

$$\tilde{W}_{(j)}^0 = 2n_j, \quad \tilde{W}_{(j)}^{t+\Delta T_{[1,\dots,m]}} = \max\{2n_j, \tilde{W}_{(j)}^t + X_{(j)}^{\Delta T_{[1,\dots,m]}} - n_j\}, \quad X_{(j)}^{\Delta T_{[1,\dots,m]}} \sim P(\lambda_j \cdot \Delta T_{[1,\dots,m]})$$

We have that $\tilde{W}_t \geq W_t, \forall t \geq 0$

Detailed proof is in Appendix ???. For the coupled process $\tilde{W}_{(j)}^t$, since the processing time of a batch containing all m types $\Delta T_{[1,\dots,m]}$ is the same under fixed thresholds $[n_1, \dots, n_m]$:

$$\Delta T_{[1,\dots,m]} = d_0 + d_1 \cdot \sum_{j=1}^m n_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right),$$

we can use batch index s for simplicity. So the coupled process is equivalent to

$$\tilde{W}_{(j)}^0 = 2n_j, \tilde{W}_{(j)}^{s+1} = \max\{2n_j, \tilde{W}_{(j)}^s + X_{(j)}^s - n_j\} = \max\{2n_j, \tilde{W}_{(j)}^s + Y_{(j)}^s\}, X_{(j)}^s \sim P(\lambda_j \cdot \Delta T_{[1, \dots, m]})$$

1. When $\Delta T_{[1, \dots, m]} = \frac{n_j}{\lambda_j}$, we have that

$$\lambda_j \cdot \Delta T = n_j \Rightarrow \mathbb{E} \left[Y_{(j)}^s \right] = 0.$$

So by Lemma ??, we have that

$$\mathbb{E} \left[W_{(j)}^S \right] \leq \mathbb{E} \left[\tilde{W}_{(j)}^S \right] \leq \lambda_j + \sqrt{\lambda_j S},$$

and Proposition ?? shows that this bound is tight.

2. When $\Delta T_{[1, \dots, m]} < \frac{n_j}{\lambda_j}$, the expectation $\mathbb{E} \left[Y_{(j)}^s \right] < 0$. The coupled process $\tilde{W}_{(j)}^s$ has a negative drift. Since

$$\text{Var}(Y_{(j)}^s) = \text{Var}(X_{(j)}^s) = \lambda_j \cdot \Delta T_{[1, \dots, m]}, \left| \mathbb{E} \left[Y_{(j)}^s \right] \right| = n_j - \lambda_j \cdot \Delta T_{[1, \dots, m]}$$

By Kingman, we have that

$$\mathbb{E} \left[W_{(j)}^S \right] \leq \mathbb{E} \left[\tilde{W}_{(j)}^S \right] \leq 2n_j + \frac{\text{Var}(Y_{(j)}^s)}{2 \left| \mathbb{E} \left[Y_{(j)}^s \right] \right|} = 2n_j + \frac{\lambda_j \cdot \Delta T_{[1, \dots, m]}}{2 \cdot (n_j - \lambda_j \cdot \Delta T_{[1, \dots, m]})}$$

So when $C = M^*$, we have that $\pi = [\frac{n_1^*}{l_{1+1}^*}, \dots, \frac{n_m^*}{l_{m+1}^*}]$. This is a special case where all types satisfy the condition $\Delta T_{[1, \dots, m]} = \frac{n_j}{\lambda_j}$. So with the union bound, we have the expected upper bound of the throughput gap to the Throughput* in (??) under the fluid system.

$$\frac{1}{S} \sum_{j=1}^m \mathbb{E} \left[W_{(j)}^S \right] \leq \frac{1}{S} \sum_{j=1}^m \lambda_j + \sum_{j=1}^m \sqrt{\frac{1}{S} \lambda_j}$$

Note that Proposition ?? shows that this bound is tight.

In the **Asymptotic Regime**,

$$\frac{1}{S^\zeta} \sum_{j=1}^m \lambda_j^\zeta + \sum_{j=1}^m \sqrt{\frac{1}{S^\zeta} \lambda_j^\zeta} = \frac{1}{\zeta S} \sum_{j=1}^m \lambda_j + \sum_{j=1}^m \sqrt{\frac{1}{\zeta S} \lambda_j} \rightarrow 0, \zeta \rightarrow \infty.$$

With the expected end-to-end latency

$$\frac{1}{S} \cdot \Delta T_{[1, \dots, m]} \cdot \left(\sum_{j=1}^m \lambda_j + \sum_{j=1}^m \sqrt{\lambda_j \cdot S} \right) \sim \Omega(\sqrt{\frac{1}{S}})$$

When all m types satisfy the condition $\Delta T_{[1, \dots, m]} < \frac{n_j}{\lambda_j}$, the expected upper bound of the throughput gap is bounded:

$$\frac{1}{S} \sum_{j=1}^m \mathbb{E} \left[W_{(j)}^S \right] \leq \frac{1}{S} \sum_{j=1}^m \left(2n_j + \frac{\lambda_j \cdot \Delta T_{[1, \dots, m]}}{2 \cdot (n_j - \lambda_j \cdot \Delta T_{[1, \dots, m]})} \right)$$

with the bounded expected end-to-end latency

$$\Delta T_{[1, \dots, m]} \cdot \frac{1}{S} \sum_{j=1}^m \mathbb{E} \left[W_{(j)}^S \right] \leq \Delta T_{[1, \dots, m]} \frac{1}{S} \sum_{j=1}^m \left(2n_j + \frac{\lambda_j \cdot \Delta T_{[1, \dots, m]}}{2 \cdot (n_j - \lambda_j \cdot \Delta T_{[1, \dots, m]})} \right) \sim \Omega(\frac{1}{S})$$

B Proof of Theorem ??

For the first segment, the state transition rule is given by

$$W_{(1)}^{s+1} = W_{(1)}^s + Y_{(1)}^s - n_1 \cdot \mathbf{1}\{W_{(1)}^s + Y_{(1)}^s \geq n_1\}, Y_{(1)}^s \sim P(\Delta T_{s\text{-th Batch}} \cdot \sum_{j=1}^m \lambda_j),$$

where s denotes the index of all batches, and $\Delta T_{s\text{-th Batch}}$ denotes the time cost of the inference of the t -th batch. Since $\Delta T_{s\text{-th Batch}} \leq \Delta T_{[1, \dots, m]}(n_1, \dots, n_m)$, where $\Delta T_{[1, \dots, m]}(n_1, \dots, n_m)$ is the time cost of the inference of the batch containing all m types, we can obtain a dominated process.

$$\bar{W}_{(1)}^{s+1} = \bar{W}_{(1)}^s + \tilde{Y}_{(1)}^s - n_1 \cdot \mathbf{1}\{\bar{W}_{(1)}^s + \tilde{Y}_{(1)}^s \geq n_1\}, Y_{(1)}^s \sim P(\Delta T_{[1, \dots, m]}(n_1, \dots, n_m) \cdot \sum_{j=1}^m \lambda_j),$$

Compared to process $\{W_{(1)}^s\}$, the process $\{\bar{W}_{(1)}^s\}$ use $Y_{(1)}^s \sim P(\Delta T_{[1, \dots, m]}(n_1, \dots, n_m) \cdot \sum_{j=1}^m \lambda_j)$ as arrival. And this process is dominated by a Lindley process

$$\tilde{W}_{(1)}^0 = 2n_1, \tilde{W}_{(1)}^{s+1} = \max\{2n_1, \tilde{W}_{(1)}^s + \tilde{Y}_{(1)}^s - n_1\}$$

Since $\Delta T_{[1, \dots, m]}(n_1, \dots, n_m) \cdot \sum_{j=1}^m \lambda_j < n_1$, the coupled process has negative drift. Since $\text{Var}(X_{(1)}^s - n_1) = \Delta T_{[1, \dots, m]}(n_1, \dots, n_m) \sum_{j=1}^m \lambda_j$ and $\left| \mathbb{E}[X_{(1)}^s - n_1] \right| = n_1 - \Delta T_{[1, \dots, m]}(n_1, \dots, n_m) \sum_{j=1}^m \lambda_j$, by Kingman Inequality, we have that

$$\mathbb{E}[W_{(1)}^s] \leq 2n_1 + \frac{\Delta T_{[1, \dots, m]} \cdot \sum_{j=1}^m \lambda_j}{2(n_1 - \Delta T_{[1, \dots, m]} \cdot \sum_{j=1}^m \lambda_j)}$$

Notice that the prompts waiting before the first segment is stored in CPU, so do not consume the GPU memory capacity, so $W_{(1)}^s$ does not affect the memory capacity.

Then we consider the k -th segment, $2 \leq k \leq m$, the arrival process is that

$$Y_{(k)}^s \sim \text{Binomial}\left(n_{k-1}, \frac{\lambda_k + \dots + \lambda_m}{\lambda_{k-1} + \lambda_k + \dots + \lambda_m}\right) = \text{Binomial}(n_{k-1}, p_k), \forall s \in \mathbb{N}$$

The k -th segment can be modeled as a process where the counting of steps is triggered by the process of the former segments, since the arrival of the k -th segment occurs if and only if the $k-1$ -th segment is processed in the former batch. And the state transition rule is given by

$$W_{(k)}^{s+1} = W_{(k)}^s + Y_{(k)}^s - n_k \cdot \mathbf{1}\{W_{(k)}^s + Y_{(k)}^s \geq n_k\}$$

where s denotes the s -th batch which contains segment $k-1$. So here the index s of each segment are indeed different, but are bounded by the total number of the batches.

We set $X_{(k)}^s = Y_{(k)}^s - n_k, \forall s \in \mathbb{N}$, and we can construct a coupled process $\tilde{W}_{(k)}^s$ to dominate $W_{(k)}^s$.

Lemma 7 Define a new process $\{\tilde{W}_{(k)}^s\}$ as a coupled version of $\{W_{(k)}^s\}$ as follows:

$$\tilde{W}_{(k)}^0 = n_k, \tilde{W}_{(k)}^{s+1} = \max\{n_k, \tilde{W}_{(k)}^s + X_{(k)}^s\}, \forall s \in \mathbb{N}$$

Then, we have that

$$\tilde{W}_{(k)}^s \geq W_{(k)}^s, \forall s \in \mathbb{N}.$$

Detailed proof is in the Appendix ??, and with the help of Lemma ??, when estimating the upper bound corresponding to $\{W_{(k)}^s\}$, we can estimate $\tilde{W}_{(k)}^s$ instead. Note that the process $\{\tilde{W}_{(k)}^s\}$ is equivalent to

$$\tilde{W}_{(k)}^0 = n_k, \tilde{W}_{(k)}^{s+1} = n_k + \max\{0, X_{(k)}^1, X_{(k)}^1 + X_{(k)}^2, X_{(k)}^1 + \dots + X_{(k)}^s\}, \forall s \in \mathbb{N}$$

Define

$$S_{(k)}^i = X_{(k)}^1 + \cdots + X_{(k)}^i$$

and we have that

$$\tilde{W}_{(k)}^0 = n_k, \tilde{W}_{(k)}^{s+1} = n_k + \max\{0, S_{(k)}^1, \dots, S_{(k)}^s\}, \forall t \geq 0, \text{ where } S_{(k)}^0 = 0$$

Next, we will estimate the $\mathbb{E} [W_{(k)}^s]$ for $2 \leq k \leq m$, since the prompts in segments $2 \rightarrow m$ consume the memory of the GPU. So we consider the expectation of $\tilde{W}_{(k)}^s$, we have that

$$\mathbb{E} [W_{(k)}^s] \leq \mathbb{E} [\tilde{W}_{(k)}^s] \leq n_k + \mathbb{E} \left[\max_{0 \leq i \leq s-1} \{S_{(k)}^i\} \right]$$

Since the expectation

$$\mathbb{E} [X_{(k)}^s] = n_{k-1} \cdot p_k - n_k < 0, \forall s \in \mathbb{N}$$

the coupled process $\tilde{W}_{(k)}^s$ has negative drift. Since $\text{Var}(X_{(k)}^i) = \text{Var}(Y_{(k)}^i) = n_{k-1} \cdot p_k (1 - p_k)$, and $|\mathbb{E} [X_{(k)}^i]| = n_k - n_{k-1} \cdot p_k$, by Kingman Inequality, we have that

$$\mathbb{E} \left[\max_{0 \leq i \leq s-1} \{S_{(k)}^i\} \right] \leq \frac{\text{Var}(X_{(k)}^i)}{2 |\mathbb{E} [X_{(k)}^i]|} \leq \frac{n_{k-1} \cdot p_k (1 - p_k)}{2 (n_k - n_{k-1} \cdot p_k)}$$

Thus, we obtain the expected upper bound:

$$\mathbb{E} [W_{(k)}^t] \leq n_k + \frac{n_{k-1} \cdot p_k (1 - p_k)}{2 (n_k - n_{k-1} \cdot p_k)}$$

The expectation bound derived above ensures that the expected queue length $\mathbb{E} [W_{(k)}^s]$ of the k -th segment is finite and controlled, which also provides a useful estimate for the expected memory consumption of the k -th segment.

As we have discussed before, the index s of each segment are different, but are all bounded by the total number of all the batches. So if we use the total number of all the batches S as the upper bound of the index s of different segments, the expected gap of the throughput $\mathbf{Throughput}^* - \mathbb{E}[\mathbf{Throughput}^{(\zeta S, \pi)}]$ is bounded by

$$\frac{1}{S} \sum_{j=1}^m \mathbb{E} [W_{(j)}^S] \leq \frac{2n_1}{S} + \frac{1}{S} \cdot \frac{\Delta T_{[1, \dots, m]} \cdot \sum_{j=1}^m \lambda_j}{2 (n_1 - \Delta T_{[1, \dots, m]} \cdot \sum_{j=1}^m \lambda_j)} + \frac{1}{S} \sum_{j=2}^m n_k + \frac{1}{S} \sum_{j=2}^m \frac{n_{k-1} \cdot p_k (1 - p_k)}{2 (n_k - n_{k-1} \cdot p_k)}$$

In the **Asymptotic Regime**, we have the estimation of $\mathbf{Throughput}^* - \mathbb{E}[\mathbf{Throughput}^{(\zeta, \pi)}]$:

$$\frac{2n_1}{\zeta S} + \frac{1}{\zeta S} \cdot \frac{\Delta T_{[1, \dots, m]} \cdot \sum_{j=1}^m \lambda_j}{2 (n_1 - \Delta T_{[1, \dots, m]} \cdot \sum_{j=1}^m \lambda_j)} + \frac{1}{\zeta S} \sum_{j=2}^m n_k + \frac{1}{\zeta S} \sum_{j=2}^m \frac{n_{k-1} \cdot p_k (1 - p_k)}{2 (n_k - n_{k-1} \cdot p_k)} = O((\zeta S)^{-1})$$

With the end-to-end expected latency and TTFT:

$$\mathbb{E} [\mathbf{Latency}^{(\zeta, \pi)}] \leq \Delta T_{[1, \dots, m]} \cdot \sum_{j=1}^m \frac{1}{\zeta S} \sum_{t=1}^T \mathbb{E} [W_t^{(j)}] \sim O(1)$$

$$\mathbb{E} [\mathbf{TTFT}^{(\zeta, \pi)}] \leq \Delta T_{[1, \dots, m]} \cdot \frac{1}{\zeta S} \sum_{t=1}^T \mathbb{E} [W_t^{(1)}] \sim O(1)$$

However, while the average is bounded, this does not prevent the queue length $W_{(k)}^s$ from exceeding this limit in specific sample paths due to stochastic fluctuations. In practical terms, there remains a positive probability of memory overflow, where $W_{(k)}^s$ grows large enough to exhaust available resources, potentially disrupting system performance. To address this, we next derive a high-probability bound to quantify the likelihood of such overflows.

As a result, we need to estimate

$$P\left(\max\{0, S_{(k)}^1, \dots, S_{(k)}^{s-1}\} \geq c\right) = P\left(\max_{0 \leq i \leq s-1} \{S_{(k)}^i\} \geq c\right), \forall c > 0, \text{ where } S_{(k)}^0 = 0$$

We construct an exponential martingale $M_{(k)}^i = e^{\theta S_{(k)}^i}$. And to make sure it is a martingale, we first consider the moment generating function

$$\phi_{(k)}(\theta) = \mathbb{E}\left[e^{\theta X_{(k)}^i}\right] = e^{-\theta n_k}(1 - p_k + p_k e^\theta)^{n_{k-1}},$$

and find $\theta > 0$ to make the moment generating function equal 1 to make sure the $M_{(k)}^i$ is a martingale. The following lemma shows that the solution $\theta > 0$ exists. And detailed proof is in the Appendix ??.

Lemma 8 Suppose $n_{k-1} > n_k > n_{k-1}p_k$ and $p_k \in (0, 1)$. Then there exists a unique solution $\theta_k > 0$ to the equation

$$e^{-\theta_k n_k}(1 - p_k + p_k e^{\theta_k})^{n_{k-1}} = 1, \quad (18)$$

where θ_k is strictly increasing with respect to $D = n_k - n_{k-1}p_k$ and satisfies the lower bound

$$\theta_k \geq \frac{8(n_k - n_{k-1}p_k)}{n_{k-1}}.$$

So when $\theta_k > 0$ satisfies $\phi(\theta_k) = 1$, $M_{(k)}^i$ is a martingale that starts at $M_{(k)}^0 = e^{\theta_k \cdot 0} = 1$.

By Doob's inequality, we have that

$$P(\max_{1 \leq i \leq s-1} \{e^{\theta_k S_{(k)}^i}\} \geq a) \leq \frac{\mathbb{E}[M^i]}{a} = \frac{\mathbb{E}[M^0]}{a} = \frac{1}{a}.$$

So we have that

$$P(\max_{1 \leq i \leq s-1} \{e^{\theta_k S_{(k)}^i}\} \geq e^{\theta_k c}) \leq \frac{1}{e^{\theta_k c}} \Rightarrow P(\max_{1 \leq i \leq s-1} \{S_{(k)}^i\} \geq c) \leq \frac{1}{e^{\theta_k c}}.$$

So consider the probability, we have

$$P(W_{(k)}^s \geq c) < P(\tilde{W}_{(k)}^s \geq c) = P(\max_{1 \leq i \leq s-1} \{S_{(k)}^i\} \geq c - n_k) \leq e^{-\theta_k(c - n_k)}$$

So for $2 \leq k \leq m, 0 \leq t \leq T$, we have that

$$P(W_{(k)}^t \geq c) < e^{-\theta_k(c - n_k)}$$

where $\theta_k > 0$ is the solution to (??).

To determine the memory lower bound, we need to ensure that the memory consumption does not exceed the bound at any $s \in \{1, \dots, S\}$ with probability at least $1 - \delta$. For each segment $k \in \{2, \dots, m\}$ and total batch index $s \in \{1, \dots, S\}$, we have:

$$P(W_{(k)}^s \geq c) < e^{-\theta_k(c - n_k)},$$

There are $m - 1$ segments (from $k = 2$ to m) and S batch counts (from $s = 1$ to S). We allocate the total failure probability δ across all segments and batch counts. The total number of events is $(m - 1) \cdot S$. Set the overflow probability for each segment and batch counts to:

$$P(W_{(k)}^s \geq c_k) \leq \frac{\delta}{(m - 1) \cdot S} \Rightarrow e^{-\theta_k(c_k - n_k)} \leq \frac{\delta}{(m - 1) \cdot S} \Rightarrow c_k \geq n_k + \frac{\ln\left(\frac{(m-1) \cdot S}{\delta}\right)}{\theta_k}$$

If we choose

$$c_k = n_k + \frac{\ln\left(\frac{(m-1) \cdot S}{\delta}\right)}{\theta_k},$$

then we have that

$$P\left(W_{(k)}^s \geq c_k\right) \leq \frac{\delta}{(m-1) \cdot S}.$$

Next, we compute the joint probability of overflow across all segments and batch counts using a union bound:

$$P\left(\bigcup_{k=2}^m \bigcup_{s=1}^S \{W_{(k)}^s \geq c_k\}\right) \leq \sum_{k=2}^m \sum_{s=1}^S P\left(W_{(k)}^s \geq c_k\right) \leq (m-1) \cdot S \cdot \frac{\delta}{(m-1) \cdot S} = \delta$$

Thus:

$$P\left(W_{(k)}^s \leq c_k \text{ for all } \begin{matrix} k \in \{2, \dots, m\}, \\ s \in \{1, \dots, S\} \end{matrix}\right) \geq 1 - \delta.$$

So the memory consumption is bounded:

$$\text{Memory Consumption}^s = M^\pi + \sum_{j=2}^m W_{(j)}^s \cdot (l + l'_{j-1}) \leq M^\pi + \sum_{j=2}^m c_j \cdot (l + l'_{j-1})$$

where M^π is the peak batch memory usage and is defined by threshold $\pi = [n_1, \dots, n_m]$

$$M^\pi = \sum_{j=1}^m n_j \left(l + \frac{l'_1 + \dots + l'_j}{2} \right) (l'_j - l'_{j-1}), l'_0 = 0$$

Substitute $c_j = n_j + \frac{\ln\left(\frac{(m-1) \cdot S}{\delta}\right)}{\theta_j}$, we have that

$$\text{Memory Consumption}^s \leq M^\pi + \sum_{j=2}^m \left(n_j + \frac{\ln\left(\frac{(m-1) \cdot S}{\delta}\right)}{\theta_j} \right) \cdot (l + l'_{j-1}).$$

Define the memory bound $\mathbf{M}^S$:

$$\mathbf{M}^S = M^\pi + \sum_{j=2}^m n_j \cdot (l + l'_{j-1}) + \sum_{j=2}^m \frac{\ln\left(\frac{(m-1) \cdot S}{\delta}\right)}{\theta_j} \cdot (l + l'_{j-1}).$$

By the lower bound in Lemma ??, we can obtain the upper bound of the third term:

$$\sum_{j=2}^m \frac{\ln\left(\frac{(m-1) \cdot S}{\delta}\right)}{\theta_j} \cdot (l + l'_{j-1}) \leq \sum_{j=2}^m \frac{n_{k-1}}{8(n_k - n_{k-1}p_k)} \ln\left(\frac{(m-1) \cdot S}{\delta}\right) \cdot (l + l'_{j-1})$$

And this $\mathbf{M}^S$ ensures that with probability at least $1 - \delta$, the memory consumption does not exceed $\mathbf{M}^S$ at any batch counts $s \in \{1, \dots, S\}$.

C Proof of Theorem ??

For the first segment, the state transition rule is

$$W_{(1)}^{s+1} = W_{(1)}^s + Y_{(1)}^s - n_1 \cdot \mathbf{1}\{W_{(1)}^s + Y_{(1)}^s \geq n_1\}, Y_{(1)}^s \sim P\left(\sum_{j=1}^m \lambda_j [t(s), t(s) + \Delta T_{s-\text{th Batch}}]\right)$$

where s denotes the total batch counts, $t(s)$ denotes the start time of s -th batch, and $\Delta T_{s\text{-th Batch}}$ denotes the cost of the inference of the s -th batch. Since $\Delta T_{s\text{-th Batch}} \leq \Delta T_{[1,\dots,m]}(n_1, \dots, n_m)$, where $\Delta T_{[1,\dots,m]}(n_1, \dots, n_m)$ is the time cost of the inference of the s -batch containing all m types, we can obtain a dominated process $\{\bar{W}_{(1)}^s\}$. Also notice that when thresholds $\pi = [n_1, \dots, n_m]$ is fixed, the time cost $\Delta T_{[1,\dots,m]}(n_1, \dots, n_m)$ is time invariant.

$$\bar{W}_{(1)}^{s+1} = \bar{W}_{(1)}^s + \bar{Y}_{(1)}^s - n_1 \cdot \mathbf{1}\{\bar{W}_{(1)}^s + \bar{Y}_{(1)}^s \geq n_1\}, Y_{(1)}^s \sim P\left(\sum_{j=1}^m \lambda_j[t(s), t(s) + \Delta T_{[1,\dots,m]}(n_1, \dots, n_m)]\right)$$

By (??), we have that

$$\sum_{j=1}^m \lambda_j[t(s), t(s) + \Delta T_{[1,\dots,m]}(n_1, \dots, n_m)] \leq \sup_{\forall s} \left\{ \sum_{j=1}^m \lambda_j[t(s), t(s) + \Delta T_{[1,\dots,m]}(n_1, \dots, n_m)] \right\} < n_1$$

We denote $\sup_{\forall s} \left\{ \sum_{j=1}^m \lambda_j[t(s), t(s) + \Delta T_{[1,\dots,m]}(n_1, \dots, n_m)] \right\}$ as $\mathbf{\Lambda}^\pi < n_1$, and we can construct a dominate process with time invariant arrival

$$\tilde{W}_{(1)}^{s+1} = \tilde{W}_{(1)}^s + \tilde{Y}_{(1)}^s - n_1 \cdot \mathbf{1}\{\tilde{W}_{(1)}^s + \tilde{Y}_{(1)}^s \geq n_1\}, \tilde{Y}_{(1)}^s \sim P(\mathbf{\Lambda}^\pi)$$

We next define that $X_{(1)}^s = \tilde{Y}_{(1)}^s - n_1$, and $\{\tilde{W}_{(1)}^s\}$ is dominated by a Lindley process

$$\bar{W}_{(1)}^0 = 2n_1, \bar{W}_{(1)}^{s+1} = \max\{2n_1, \bar{W}_{(1)}^s + X_{(1)}^s\}, \forall s \in \mathbb{N}$$

Then we have that

$$W_{(1)}^s \leq \bar{W}_{(1)}^s \leq \tilde{W}_{(1)}^s \leq \bar{W}_{(1)}^s, \forall s \in \mathbb{N}.$$

Since $n_1 > \mathbf{\Lambda}^\pi$, the process $\{\bar{W}_{(1)}^s\}$ has negative drift. Since

$$\text{Var}(X_{(1)}^s) = \mathbf{\Lambda}^\pi, \left| \mathbb{E} \left[X_{(1)}^s \right] \right| = n_1 - \mathbf{\Lambda}^\pi$$

By Kingman Inequality, we have that

$$\mathbb{E} \left[W_{(1)}^s \right] \leq 2n_1 + \frac{\mathbf{\Lambda}^\pi}{2(n_1 - \mathbf{\Lambda}^\pi)}$$

Next we consider the k -th segment, $2 \leq k \leq m$. The arrival for the k -th segment is from the last batch containing the $k-1$ -th segment, so the index s to the k -th segment denotes the batch counts of the batch containing the $k-1$ -th batch. Notice that, the index s to each segment are different, but are bounded by the total batch counts.

Now consider the s -th arrival of the k -th segment, it comes from the s -th batch containing the $k-1$ -th batch and the parameter $p_k(s)$ is defined by the components of this batch. Although we do not exactly know the arrival time of these prompts, but we know that the arrival happens from t to $t + \Delta t$, where $t < t + \Delta t \leq t(s, k-1)$, $t(s, k-1)$ is the starting time of the s -th batch containing the $k-1$ -th segment. By (??), we define that

$$p_k(s) = \frac{\sum_{j=k+1}^m \lambda_j[t, t + \Delta t]}{\sum_{j=k}^m \lambda_j[t, t + \Delta t]} \leq p_k^* < \frac{n_k}{n_{k-1}}, \forall s \in \mathbb{N}.$$

So the arrival process is

$$Y_{(k)}^s \sim \text{Binomial}(n_{k-1}, p_k(s)), \forall s \geq 0$$

So the state transition rule is given by

$$W_{(k)}^{s+1} = W_{(k)}^s + Y_{(k)}^s - n_k \cdot \mathbf{1}\{W_{(k)}^s + Y_{(k)}^s \geq n_k\}$$

where s denotes the s -th batch which contains segment $s-1$, since there is new arrival to segment s if and only if the segment $s-1$ is contained in the former batch.

We set $X_{(k)}^s = Y_{(k)}^s - n_k, \forall s \in \mathbb{N}$. Although here the distribution of $Y_{(k)}^s \sim \text{Binomial}(n_{k-1}, p_k(s))$ is time-varying, the coupling construction is still the same as Lemma ??

Lemma 9 Define a new process $\{\tilde{W}_{(k)}^s\}$ as a coupled version of $\{W_{(k)}^s\}$ as follows:

$$\tilde{W}_{(k)}^0 = n_k, \tilde{W}_{(k)}^{s+1} = \max\{n_k, \tilde{W}_{(k)}^s + X_{(k)}^s\}, \forall t \geq 0, X_{(k)}^s = Y_{(k)}^s - n_1.$$

Then, we have that

$$\tilde{W}_{(k)}^s \geq W_{(k)}^s, \forall s \in \mathbb{N}$$

However, in order to use the Kingman , we should construct another coupled process with time-invariant arrival. So consider

$$\bar{W}_{(k)}^0 = n_k, \bar{W}_{(k)}^{s+1} = \max\{n_k, \bar{W}_{(k)}^s + \bar{X}_{(k)}^s\}, \bar{X}_{(k)}^s = \bar{Y}_{(k)}^s - n_k, \bar{Y}_{(k)}^s \sim \text{Binomial}(n_{k-1}, p_k^*), \forall s \in \mathbb{N}.$$

Then we have that

$$\bar{W}_{(k)}^s \geq \tilde{W}_{(k)}^s \geq W_{(k)}^s, \forall s \in \mathbb{N}$$

Note that the process $\bar{W}_{(k)}^s$ is equivalent to

$$\bar{W}_{(k)}^0 = n_k, \bar{W}_{(k)}^{s+1} = n_k + \max_{0 \leq i \leq t} \{\bar{S}_{(k)}^i\}, \text{ where } \bar{S}_{(k)}^0 = 0, \bar{S}_{(k)}^i = \sum_{j=1}^i \bar{X}_{(k)}^j, 1 \leq i \leq s$$

Since the expectation

$$\mathbb{E} [\bar{X}_{(k)}^s] = n_{k-1} \cdot p_k^* - n_k < 0, \forall t \in \mathbb{N}$$

the coupled process $\{\bar{W}_{(k)}^s\}$ has a negative drift. Since

$$\text{Var}(\bar{X}_{(k)}^s) = n_{k-1} p_k^* (1 - p_k^*), \left| \mathbb{E} [\bar{X}_{(k)}^s] \right| = n_k - p_k^* n_{k-1}$$

by Kingman, we have that

$$\mathbb{E} \left[\max_{0 \leq i \leq t} \{\bar{S}_{(k)}^i\} \right] \leq \frac{n_{k-1} p_k^* (1 - p_k^*)}{2(n_k - n_{k-1} p_k^*)}$$

Thus, we obtain the expected upper bound:

$$\mathbb{E} [W_{(k)}^s] \leq \mathbb{E} [\tilde{W}_{(k)}^s] \leq \mathbb{E} [\bar{W}_{(k)}^s] \leq n_k + \frac{n_{k-1} p_k^* (1 - p_k^*)}{2(n_k - n_{k-1} p_k^*)}$$

The discussion of the asymptotic optimal throughput under bounded latency and TTFT is the same as the discussion in Appendix ???. For the high probability bound, it is also similar to the discussion in Appendix ??? considering the coupled process $\{\bar{W}_{(k)}^s\}$. Recall the definition of $\{\bar{W}_{(k)}^s\}$:

$$\bar{W}_{(k)}^0 = n_k, \bar{W}_{(k)}^{s+1} = n_k + \max_{0 \leq i \leq t} \{\bar{S}_{(k)}^i\}, \text{ where } \bar{S}_{(k)}^0 = 0, \bar{S}_{(k)}^i = \sum_{j=1}^i \bar{X}_{(k)}^j, 1 \leq i \leq s$$

where $\bar{X}_{(k)}^j = \bar{Y}_{(k)}^j - n_k, \bar{Y}_{(k)}^j \sim \text{Binomial}(n_{k-1}, p_k^*)$ and $p_k^* < \frac{n_k}{n_{k-1}}$. The same to the discussion in Appendix ??? after the Lemma ???, the memory bound M_S here is:

$$\mathbf{M}_S = M^\pi + \sum_{j=2}^m n_j \cdot (l + l'_{j-1}) + \sum_{j=2}^m \theta_j^{-1} \ln \left(\frac{(m-1) \cdot S}{\delta} \right) \cdot (l + l'_{j-1})$$

where $\theta_j, 2 \leq j \leq m$ here is the solution to the equation similar to (??)

$$e^{-\theta_j n_j} (1 - p_j^* + p_j^* e^{\theta_j})^{n_{j-1}} = 1 \quad (19)$$

Since

$$p_j^* = \arg \min_{0 < p_j < 1} \{n_j - p_j n_{j-1}\},$$

by the strictly increasing property of θ_j in Lemma ???, the corresponding θ_j is the smallest solution under different $0 < p_j < 1$, thus obtain the biggest term $\sum_{j=2}^m \theta_j^{-1} \ln \left(\frac{(m-1) \cdot (S+1)}{\delta} \right) \cdot (l + l'_{j-1})$. And the following discussion considering the lower bound of θ_j is the same in Appendix ???.

D Proof of Lemmas

D.1 Proof of Lemma ??

We take the expectation on both sides of the state transition equation

$$\mathbb{E} [W^{t+1}] = \mathbb{E} [W^t] + \lambda - \lambda \mathbb{P}(W^t + X^t \geq \lambda)$$

Summing from $t = 0$ to $T - 1$, and noting that $W_0 = 0$, we get the lemma

$$\mathbb{E} [W^T] = \lambda T - \lambda \mathbb{E} [T - T_{\text{stuck}}] = \lambda \cdot \mathbb{E} [T_{\text{stuck}}]$$

D.2 Proof of Lemma ??

We prove this by induction. First, for $t = 0$, we have $\tilde{W}^0 = 2\lambda \geq \lambda = W^0 + \lambda$. Next, assume that $\tilde{W}^t \geq W^t + \lambda$ holds for some $t \geq 0$. We now prove that $\tilde{W}^{t+1} \geq W^{t+1} + \lambda$. Consider the following two cases:

1. If $W^t + X^t \geq \lambda$, then $W^{t+1} = W^t + X^t - \lambda$. In this case,

$$\tilde{W}^{t+1} = \max\{2\lambda, \tilde{W}^t + X^t - \lambda\} \geq \tilde{W}^t + X^t - \lambda \geq (W^t + \lambda) + X^t - \lambda = W^{t+1} + \lambda.$$

2. If $W^t + X^t < \lambda$, then $W^{t+1} = W^t + X^t$. In this case,

$$\tilde{W}^{t+1} = \max\{2\lambda, \tilde{W}^t + X^t - \lambda\} \geq 2\lambda = \lambda + \lambda > W^{t+1} + \lambda.$$

Hence, $\tilde{W}^{t+1} \geq W^{t+1} + \lambda$. By the principle of induction, we have $\tilde{W}^t \geq W^t + \lambda$ for all $t \in \mathbb{N}$.

D.3 Proof of Lemma ??

Since $\tilde{W}_{(j)}^t \geq 2n_j$, when $W_{(j)}^t < n_j$, we have $\tilde{W}_{(j)}^t \geq W_{(j)}^t$.

At first, $W_{(j)}^0 = 0$, but $W_{(j)}^t$ will achieve n_j and type j starts to join the batch. And after a period of time, $W_{(j)}^t$ drops back below n_j , and repeats the process above.

In each period $T_0 \leq t \leq T_1$ when $W_{(j)}^t \geq n_j$, we set $\tau = \max\{t \leq T_0 | W_{(j)}^t = n_j\}$. So $\forall t \in [T_0, T_1]$, the number of iterations of $W_{(j)}^t$ and $\tilde{W}_{(j)}^t$ from τ to t is that

$$I_{\tau \rightarrow t} \geq \lfloor \frac{t - \tau}{\Delta T_{[1, \dots, m]}} \rfloor, \tilde{I}_{\tau \rightarrow t} \leq 1 + \lfloor \frac{t - \tau}{\Delta T_{[1, \dots, m]}} \rfloor \Rightarrow \tilde{I}_{\tau \rightarrow t} \leq 1 + I_{\tau \rightarrow t}.$$

So the coupled process at most makes one more iteration than the real process after τ . For example, at τ , the real process is just at a batch inference without type j , but the coupled process just finished a batch inference with all the types, which yields a n_j drop on $\tilde{W}_{(j)}^t$. So at τ , $\tilde{W}_{(j)}^t \geq n_j = W_{(j)}^t$. And for time $(\tau, t]$, since the arrival dynamic is the same between the two processes, at the real process type j joins every batch, and the batch may not contain all types compared with the coupled process. So the numbers of the finish of a batch of coupled process is at most the same as the real ones. So $\tilde{W}_{(j)}^t \geq W_{(j)}^t$ holds during $(\tau, t]$, hence $\forall t \geq 0$.

D.4 Proof of Lemma ??

We prove the dominance by induction. When $t = 0$, it is obvious that $\tilde{W}_{(k)}^0 = n_k \geq 0 = W_{(k)}^t$. Next, we assume that $\tilde{W}_{(k)}^t \geq W_{(k)}^t$ holds for some $t \geq 0$. We now prove that $\tilde{W}_{(k)}^{t+1} \geq W_{(k)}^{t+1}$. Consider the following two cases:

1. If $W_{(k)}^t + Y_{(k)}^t \geq n_k$, then $W_{(k+1)}^t = W_{(k)}^t + Y_{(k)}^t - n_k$. In this case,

$$\tilde{W}_{(k)}^{t+1} = \max\{n_k, \tilde{W}_{(k)}^t + X_{(k)}^t\} \geq \tilde{W}_{(k)}^t + X_{(k)}^t = W_{(k)}^t + Y_{(k)}^t - n_k = W_{(k+1)}^t$$

2. If $W_{(k)}^t + Y_{(k)}^t < n_k$, then $W_{(k+1)}^t = W_{(k)}^t + Y_{(k)}^t$. In this case,

$$\tilde{W}_{(k)}^{t+1} = \max\{n_k, \tilde{W}_{(k)}^t + X_{(k)}^t\} \geq n_k > W_{(k+1)}^t$$

D.5 Proof of Lemma ??

First, we prove that the solution $\theta_k > 0$ exists and is unique and for simplicity, we denote $\theta = \theta_k$ here. Define the function

$$f(\theta) = e^{-\theta n_k} (1 - p_k + p_k e^\theta)^{n_{k-1}}.$$

We need to show that there exists $\theta > 0$ such that $f(\theta) = 1$. First, evaluate $f(\theta)$ at $\theta = 0$:

$$f(0) = e^0 (1 - p_k + p_k \cdot 1)^{n_{k-1}} = 1.$$

Thus, $\theta = 0$ is a solution, but we seek the solution $\theta > 0$. Next, we consider the logarithm of $f(\theta)$

$$g(\theta) = \ln f(\theta) = -\theta n_k + n_{k-1} \ln(1 - p_k + p_k e^\theta),$$

$$g'(\theta) = -n_k + n_{k-1} \cdot \frac{p_k e^\theta}{1 - p_k + p_k e^\theta}.$$

At $\theta = 0$,

$$g(0) = 0, g'(0) = -n_k + n_{k-1} p_k.$$

Since $\mathbb{E}[X_{(k)}^s] = n_{k-1} p_k - n_k < 0$, we have $g'(0) < 0$, so $f(\theta)$ is decreasing near $\theta = 0$. Consider the second derivative:

$$g''(\theta) = n_{k-1} \cdot \frac{p_k e^\theta (1 - p_k)}{(1 - p_k + p_k e^\theta)^2} > 0 \quad \text{for } \theta > 0,$$

since $p_k \in (0, 1)$ and $n_{k-1} > 0$. Thus, $g(\theta)$ is convex, and so is $f(\theta)$. Now, consider the limit as $\theta \rightarrow \infty$:

$$f(\theta) \approx p_k^{n_{k-1}} e^{\theta(n_{k-1} - n_k)}.$$

By our settings, $n_{k-1} > n_k$, this ensures that $f(\theta) \rightarrow \infty, \theta \rightarrow \infty$. Thus, $f(\theta)$ is continuous, convex, starts at $f(0) = 1$, decreases below 1 for small $\theta > 0$, and increases to infinity. By the intermediate value theorem, there exists only one $\theta > 0$ such that $f(\theta) = 1$.

Next, we prove the monotonicity of the solution $\theta > 0$ with respect to $D = n_k - n_{k-1} p_k$. Recall that $g(0) = 0$, $g'(0) = -D < 0$, and $g''(\theta) > 0$ for $\theta > 0$, so $g(\theta)$ is strictly convex. The unique positive solution $\theta > 0$ occurs where $g(\theta)$ crosses zero after initially decreasing from $g(0) = 0$. Consider two values D_1 and D_2 such that $D_1 < D_2$, with corresponding functions $g_1(\theta) = -\theta(n_{k-1} p_k + D_1) + n_{k-1} \ln(1 - p_k + p_k e^\theta)$ and $g_2(\theta) = -\theta(n_{k-1} p_k + D_2) + n_{k-1} \ln(1 - p_k + p_k e^\theta)$, and solutions θ_1 and θ_2 , respectively.

At $\theta = 0$, $g'_1(0) = -D_1 > g'_2(0) = -D_2$, meaning $g_2(\theta)$ decreases more steeply from zero than $g_1(\theta)$. For a fixed $\theta > 0$, the partial derivative $\frac{\partial g}{\partial D} = -\theta < 0$. Thus, increasing D decreases $g(\theta)$ pointwise. Since $g(\theta)$ is convex and approaches infinity as $\theta \rightarrow \infty$, the zero crossing shifts to a larger θ to restore $g(\theta; D) = 0$. Hence, if $D_2 > D_1$, then $\theta_2 > \theta_1$.

Finally, we provide asymptotic behavior and bounds for the solution $\theta > 0$ in terms of D , n_{k-1} , and p_k . When D is small, θ is also small. Expand $g(\theta)$ around $\theta = 0$, we have that $g(\theta) = g(0) + g'(0)\theta + \frac{1}{2}g''(0)\theta^2 + O(\theta^3) = 0 - D\theta + \frac{1}{2}n_{k-1}p_k(1 - p_k)\theta^2 + O(\theta^3)$. Setting $g(\theta) = 0$ and neglecting higher-order terms, we obtain that $-D\theta + \frac{1}{2}n_{k-1}p_k(1 - p_k)\theta^2 \approx 0 \implies \theta \approx \frac{2D}{n_{k-1}p_k(1 - p_k)}$. Thus, for small D , $\theta = O(D)$, and specifically, $\theta \approx \frac{2(n_k - n_{k-1}p_k)}{n_{k-1}p_k(1 - p_k)}$.

Next we apply Taylor's theorem with remainder to find the general lower bound of the solution $\theta > 0$.

$$g(\theta) = g(0) + g'(0)\theta + \frac{1}{2}g''(c)\theta^2 = -D\theta + \frac{1}{2}g''(c)\theta^2 = 0 \implies \theta = \frac{2D}{g''(c)}. \quad \text{for some } c \in (0, \theta)$$

So we need to find an upper bound for $g''(c)$, since $g''(x) = n_{k-1} \cdot \frac{p_k e^x (1 - p_k)}{(1 - p_k + p_k e^x)^2}$, we define $h(y) = \frac{p_k y (1 - p_k)}{(1 - p_k + p_k y)^2}$ where $y = e^x \geq 1$. The function $h(y)$ achieves a maximum of $\frac{1}{4}$ over $y \geq 1$ and $0 < p_k < 1$. Thus we have that $g''(x) \leq n_{k-1} \cdot \frac{1}{4} = \frac{n_{k-1}}{4} \implies \theta = \frac{2D}{g''(c)} \geq \frac{2D}{n_{k-1}/4} = \frac{8D}{n_{k-1}}$. So a general lower bound is:

$$\theta \geq \frac{8(n_k - n_{k-1}p_k)}{n_{k-1}}. \tag{20}$$